

Nama : Kolonel Gallih Bimantara

NPM : 23312199

Jobdesk : api, user

DOKUMENTASI

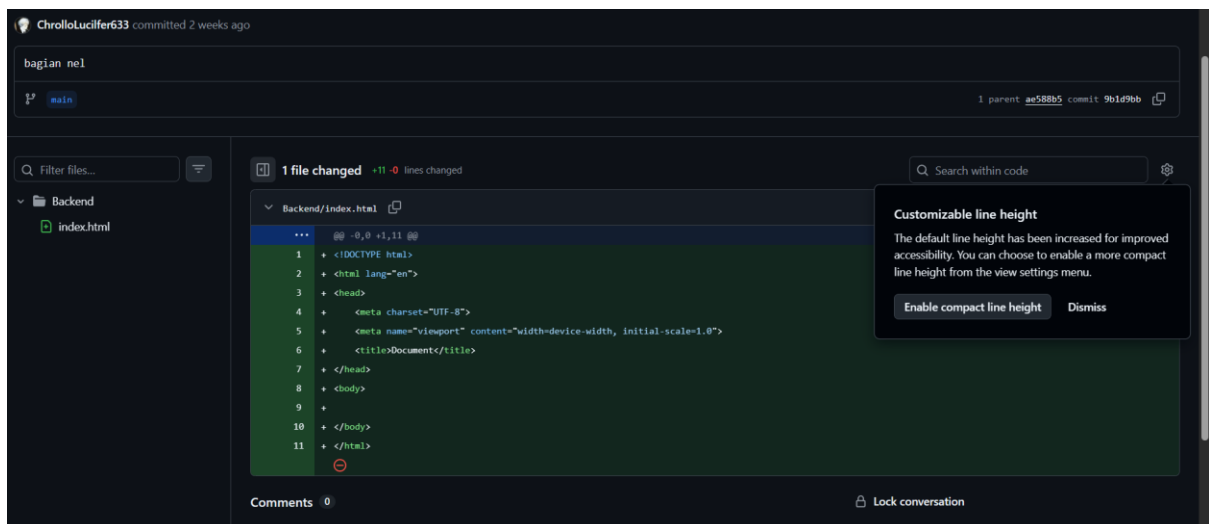
1. Commits on April 4, 2026



Pada tahap ini saya melakukan inisialisasi repositori untuk pertama kalinya saya mengunggah file dasar yang akan digunakan dalam proyek.

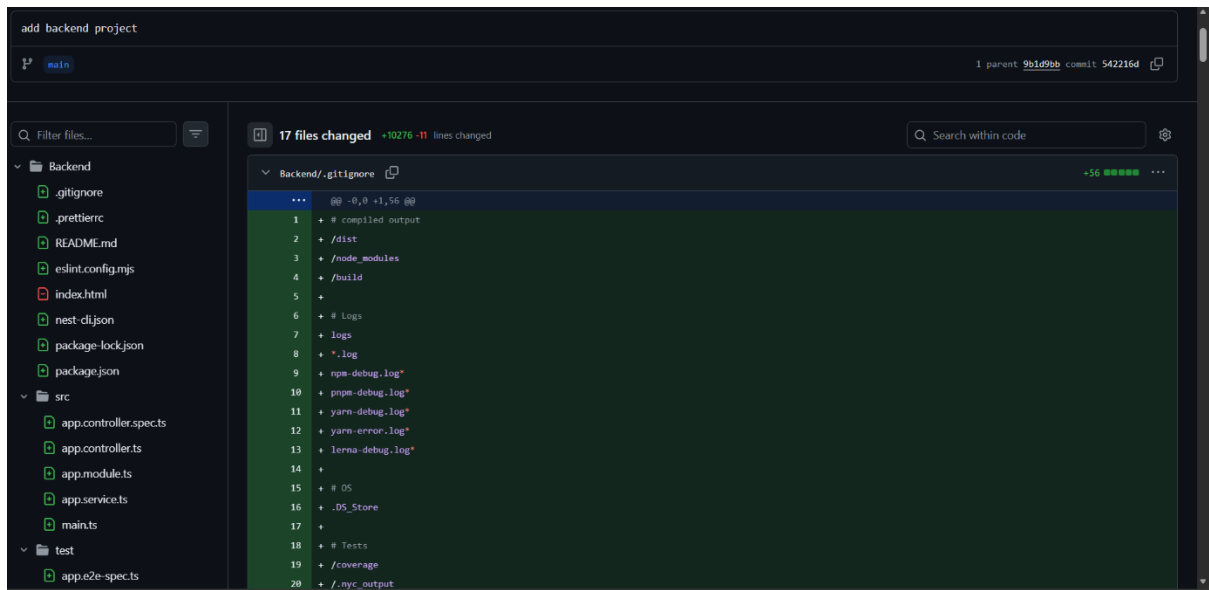
2. Commits on April 5, 2026

a. Bagian kolonel



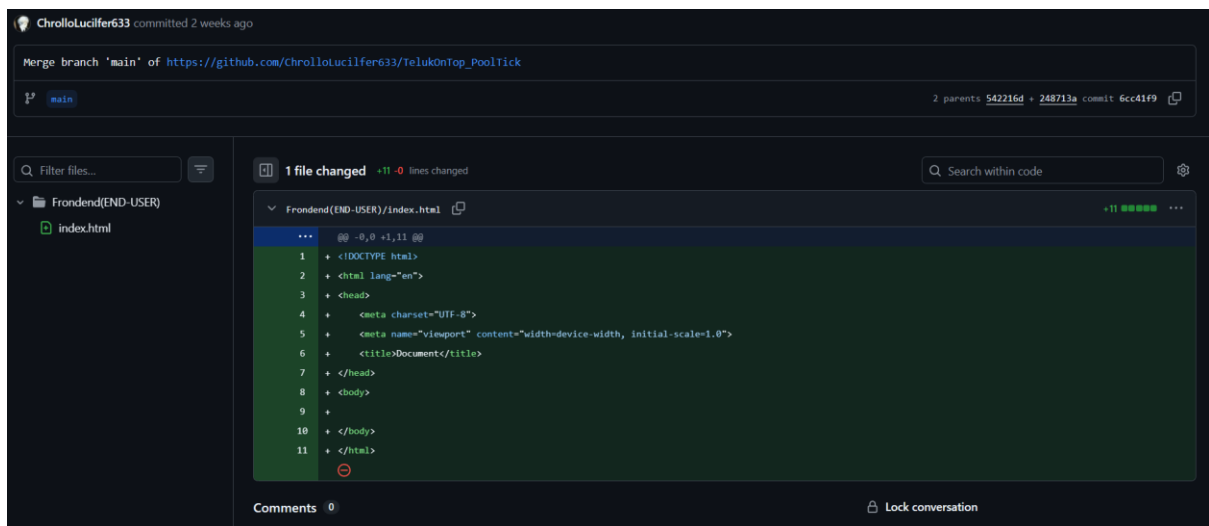
Pada tahap ini, saya membuat file index html dulu di dalam folder backend

b. Add backend project



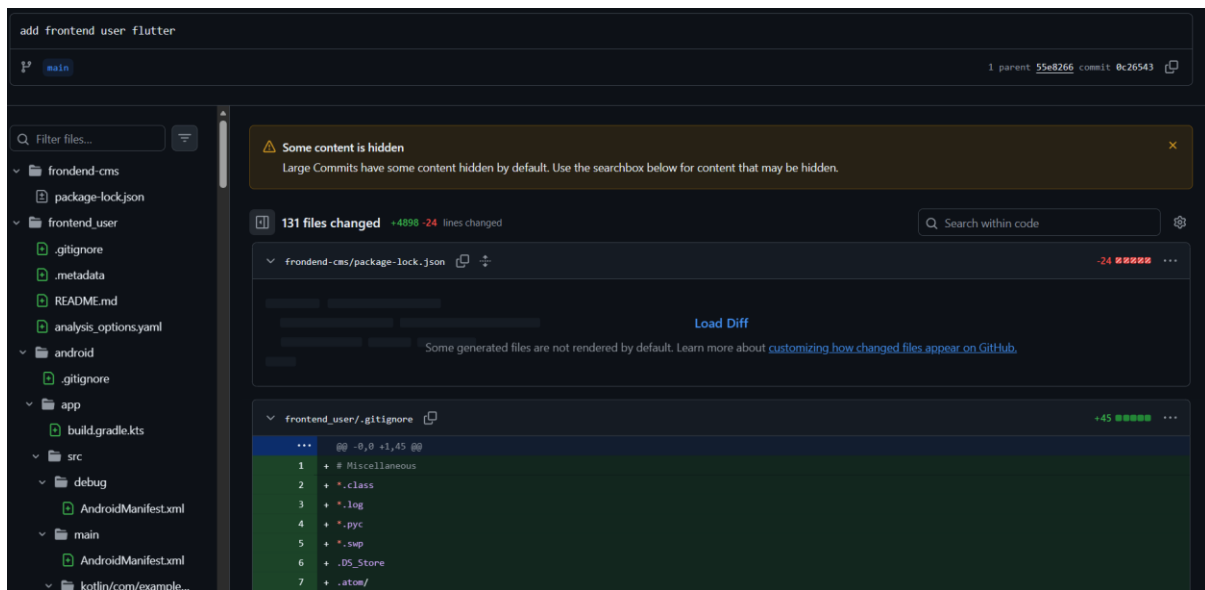
mengonfigurasi file .gitignore untuk memastikan file sampah atau folder berat seperti node_modules tidak ikut terunggah ke repositori, selain itu, saya menyiapkan struktur folder standar Node.js yang mencakup file konfigurasi seperti package.json, eslint, dan prettier guna menjaga kualitas dan konsistensi kode program

c. Merge branch 'main' of https://github.com/ChrolloLucifer633/TelukOnTop_PoolTick



Disini saya melakukan merge branch untuk menggabungkan perubahan terbaru ke cabang utama. Dalam proses ini, saya menambahkan struktur dasar Frontend(END-USER)

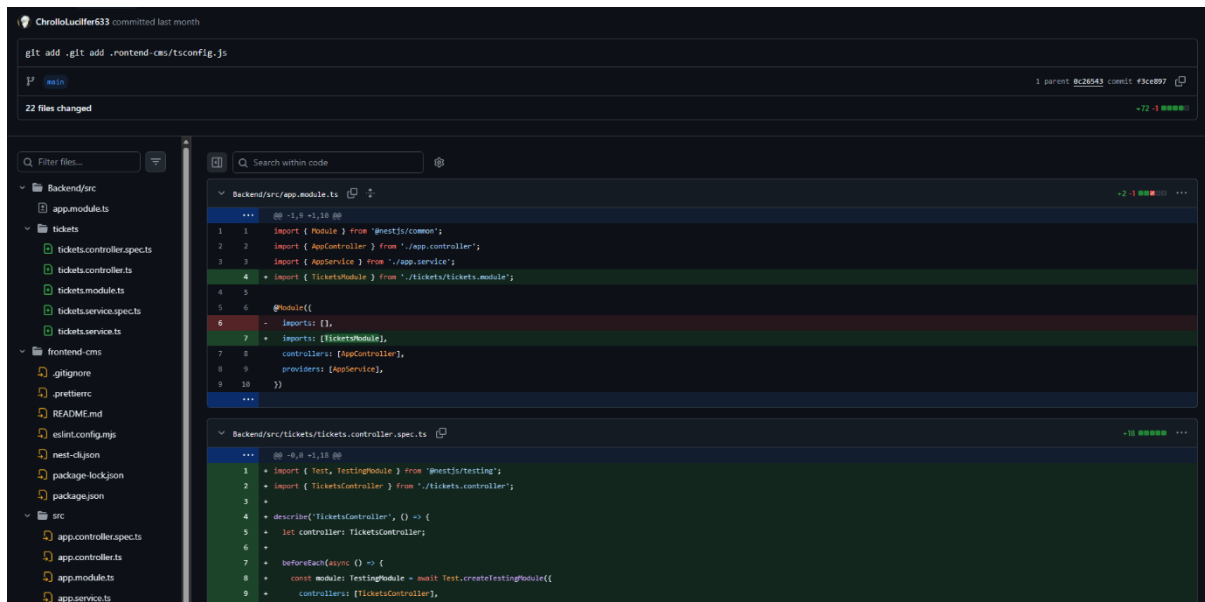
d. add frontend user flutter



Pada tahap ini, saya menambahkan proyek Frontend User menggunakan Flutter framework saya menginisialisasi struktur aplikasi mobile yang mencakup konfigurasi untuk platform Android, file analysis_options.yaml untuk standarisasi kode, serta file .gitignore khusus Flutter guna memastikan file build system tidak mengotori repostori. Langkah ini bertujuan untuk memulai pengembangan antarmuka pengguna berbasis aplikasi mobile

3. Commits on April 6, 2026

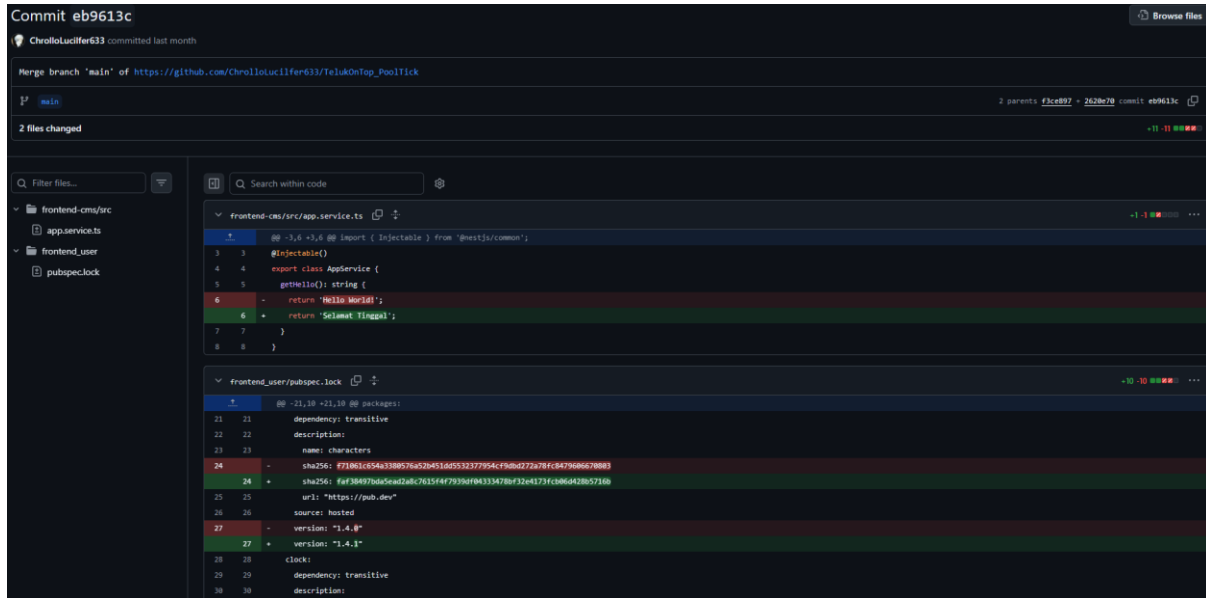
a. [git add .git add .rontend-cms/tsconfig.js](#)



Pada bagian ini, saya mengembangkan fitur Tickets di sisi Backend menggunakan NestJS. Saya membuat modul baru bernama TicketsModule dan mendaftarkannya ke dalam AppModule agar fitur tersebut dapat berjalan di dalam aplikasi. Selain itu, saya juga menyusun

TicketsController beserta file pengujiannya (spec.ts) untuk menangani logika API terkait sistem pertiketan secara terstruktur

b. Merge branch 'main'
of https://github.com/ChrolloLucifer633/TelukOnTop_PoolTick



```
Commit eb9613c
ChrolloLucifer633 committed last month

Merge branch 'main' of https://github.com/ChrolloLucifer633/TelukOnTop_PoolTick

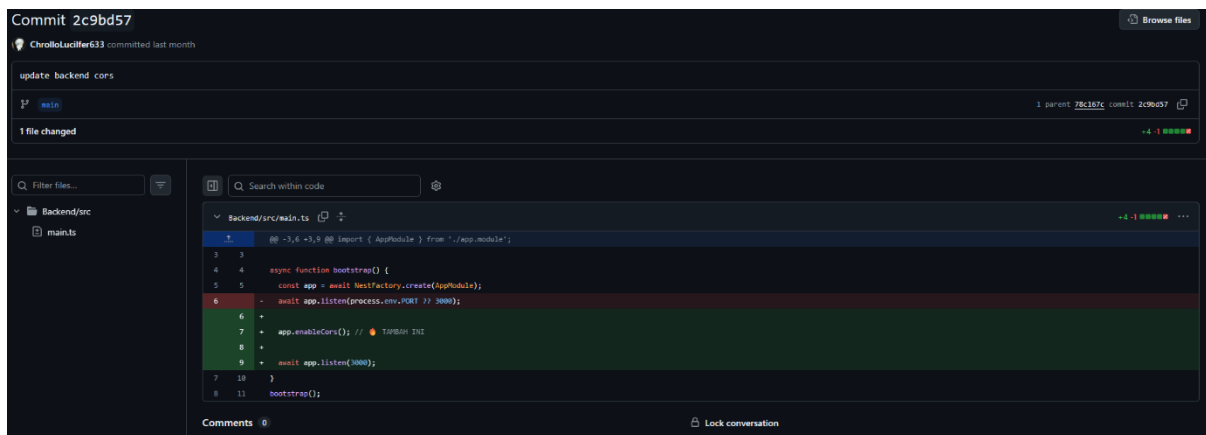
2 files changed

frontend-cms/src/app.service.ts
3 3  @Injectable()
4 4  export class AppService {
5 5  getHello(): string {
6 -   return 'Hello World!';
6 +   return 'Selamat Tinggal';
7 7  }
8 8  }

frontend_user/pubspec.lock
21 21  dependency: transitive
22 22  description:
23 23  name: characters
24 - sha256: 72161c54a3389c76452645164532377954c19b4d772a78f6947968670883
24 + sha256: f4f384973da5ead2a8763564f79394f043354780f3244173f1386d4380577169
25 25  url: "https://pub.dev"
26 26  source: hosted
27 - version: "1.4.0"
27 + version: "1.4.1"
```

pada commit ini, saya melakukan update pesan service dan pembaruan dependensi. Di bagian backend, saya mengubah *return value* pada fungsi getHello() di file app.service.ts dari 'Hello World!' menjadi 'Selamat Tinggal'. Selain itu, saya melakukan pembaruan versi package pada aplikasi Flutter melalui file pubspec.lock (dari versi 1.4.0 ke 1.4.1) untuk memastikan dependensi proyek tetap mutakhir

c. update backend cors



```
Commit 2c9bd57
ChrolloLucifer633 committed last month

update backend cors

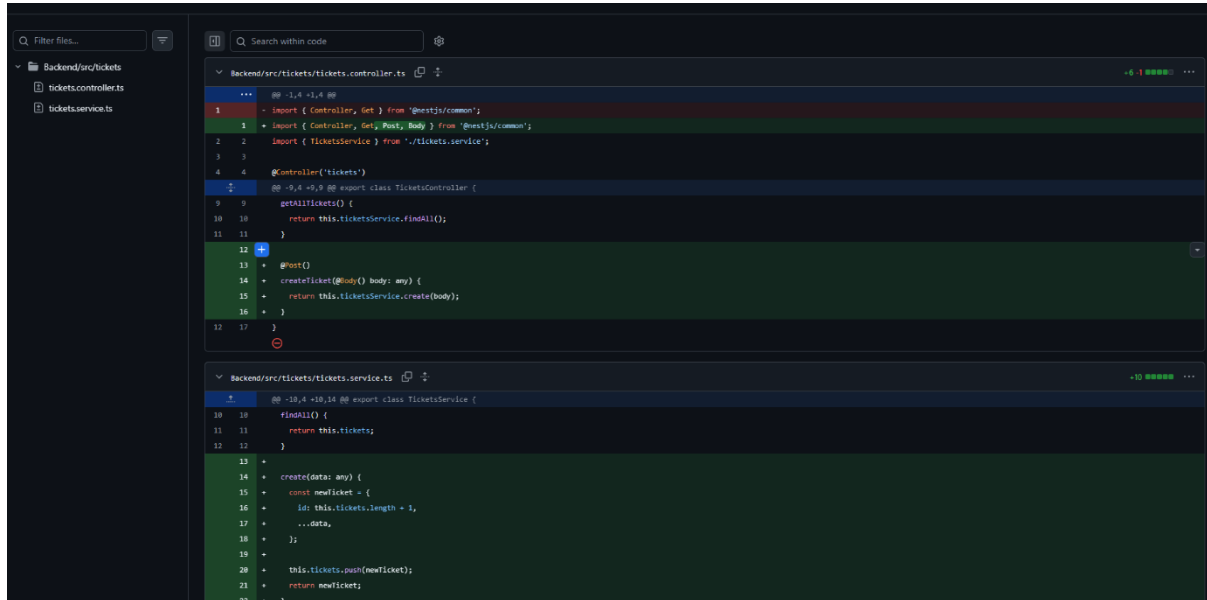
1 file changed

Backend/src/main.ts
3 3  @Injectable()
4 4  export class AppService {
5 5  bootstrap() {
6 -   const app = await NestFactory.create(AppModule);
6 +   const app = await NestFactory.create(AppModule);
7 +   app.enableCors();
8 +   await app.listen(process.env.PORT ?? 3000);
9 +   bootstrap();
10 10  }
11 11  }
12 12  bootstrap();
```

Di sini saya melakukan konfigurasi CORS (Cross-Origin Resource Sharing) pada file main.ts di sisi Backend. Saya menambahkan perintah app.enableCors() agar server NestJS dapat menerima permintaan (request) dari domain luar, seperti aplikasi Frontend atau Mobile yang

sudah dibuat sebelumnya. Selain itu, saya juga menetapkan port aplikasi secara statis ke port 3000 untuk memastikan koneksi antar layanan berjalan stabil

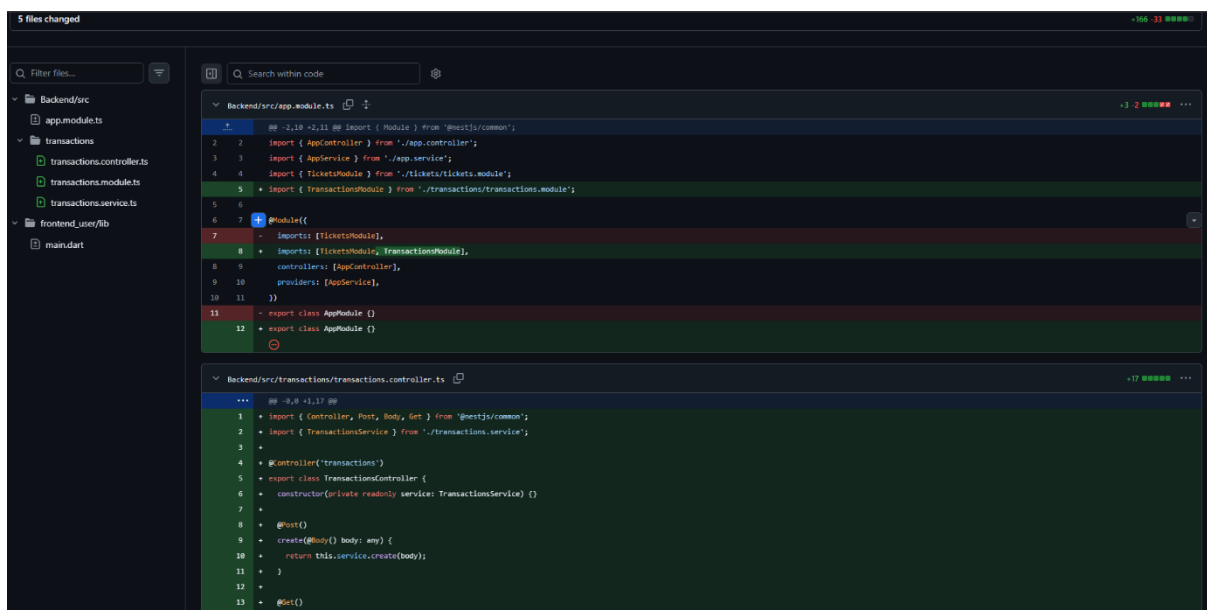
d. Tambah fitur POST tiket



```
Backend/src/tickets/tickets.controller.ts
1 // @ts-nocheck
2 import { Controller, Get } from '@nestjs/common';
3 import { Controller, Get, Post, Body } from '@nestjs/common';
4 import { TicketsService } from './tickets.service';
5
6 @Controller('tickets')
7 export class TicketsController {
8   constructor(private ticketsService: TicketsService) {}
9
10  @Get()
11  getAllTickets() {
12    return this.ticketsService.findAll();
13  }
14
15  @Post()
16  createTicket(@Body() body: any) {
17    return this.ticketsService.create(body);
18  }
19 }
20
21 Backend/src/tickets/tickets.service.ts
22 // @ts-nocheck
23 import { Injectable } from '@nestjs/common';
24
25 @Injectable()
26 export class TicketsService {
27   tickets: any[] = [];
28
29   findAll() {
30     return this.tickets;
31   }
32
33   create(data: any) {
34     const newTicket = {
35       id: this.tickets.length + 1,
36       ...data,
37     };
38     this.tickets.push(newTicket);
39     return newTicket;
40   }
41 }
```

Pada tahap ini, saya menambahkan fitur pembuatan tiket baru pada sisi Backend. Di dalam tickets.controller.ts, saya menambahkan dekorator `@Post()` dan method `createTicket` untuk menangani permintaan data dari user. Secara bersamaan, saya juga memperbarui tickets.service.ts dengan logika untuk memproses data tersebut, di mana sistem akan otomatis membuat ID unik berdasarkan panjang data yang ada dan menyimpannya ke dalam array tickets

e. Tambah fitur transaksi + flutter beli tiket

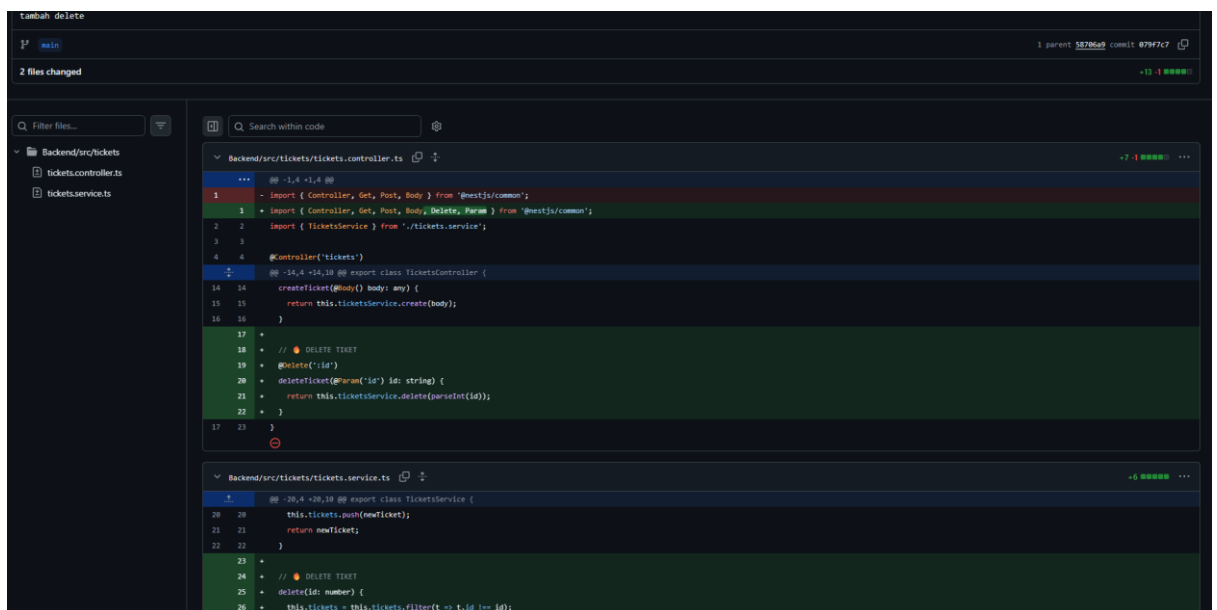


```
Backend/src/app.module.ts
1 // @ts-nocheck
2 import { Module } from '@nestjs/common';
3 import { AppController } from './app.controller';
4 import { AppService } from './app.service';
5 import { TicketsModule } from './tickets/tickets.module';
6 import { TransactionsModule } from './transactions/transactions.module';
7
8 @Module({
9   imports: [TicketsModule, TransactionsModule],
10   controllers: [AppController],
11   providers: [AppService],
12 })
13 export class AppModule {}
14
15 Backend/src/transactions/transactions.controller.ts
16 // @ts-nocheck
17 import { Controller, Post, Get } from '@nestjs/common';
18 import { TransactionsService } from './transactions.service';
19
20 @Controller('transactions')
21 export class TransactionsController {
22   constructor(private readonly service: TransactionsService) {}
23
24   @Post()
25   createTransaction(@Body() body: any) {
26     return this.service.create(body);
27   }
28
29   @Get()
30   getAllTransactions() {
31     return this.service.findAll();
32   }
33 }
```

Pada tahap ini, saya mengimplementasikan fitur Transactions pada sisi Backend. Saya membuat folder baru bernama transactions yang berisi TransactionsModule, TransactionsController, dan TransactionsService. Saya juga mendaftarkan TransactionsModule ke dalam AppModule agar fitur transaksi ini aktif di sistem. Controller yang baru dibuat ini sudah mendukung method `@Post()` untuk membuat transaksi baru dan `@Get()` untuk mengambil data transaksi yang ada

4. Commits on April 8, 2026

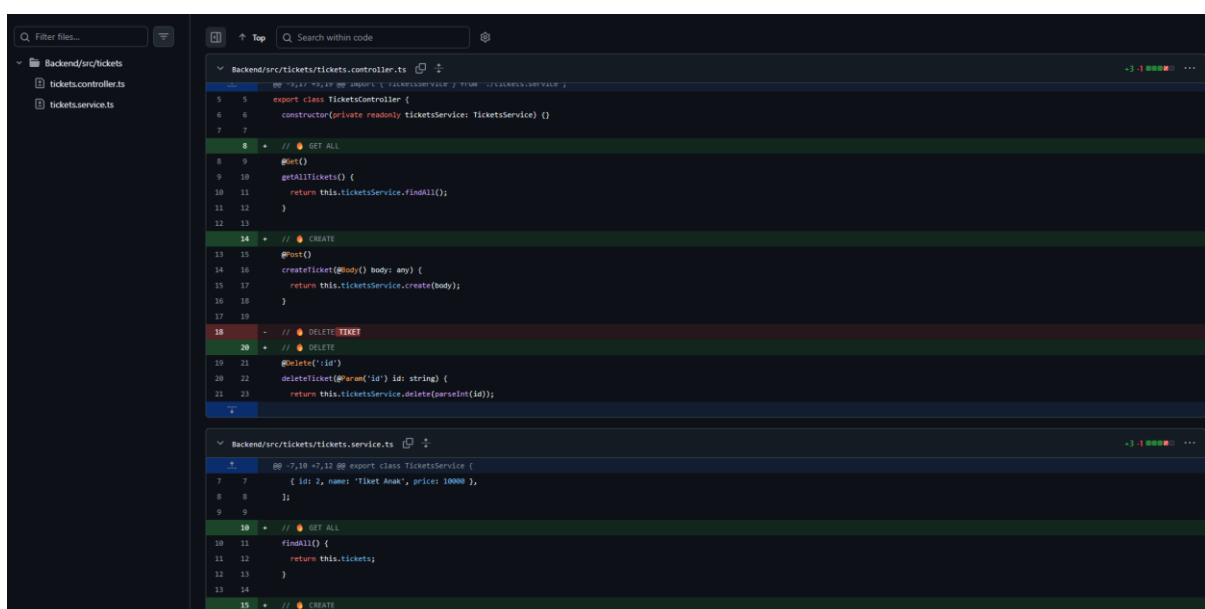
a. Tambah delete



```
Backend/src/tickets/tickets.controller.ts
1  import { Controller, Get, Post, Body } from '@nestjs/common';
2  import { Controller, Get, Post, Delete, Param } from '@nestjs/common';
3  import { TicketsService } from './tickets.service';
4
5  @Controller('tickets')
6  export class TicketsController {
7    constructor(private ticketsService: TicketsService) {}
8
9    // DELETE TICKET
10    @Delete(':id')
11    deleteTicket(@Param('id') id: string) {
12      return this.ticketsService.delete(parseInt(id));
13    }
14  }
15
16 Backend/src/tickets/tickets.service.ts
17
18 // DELETE TICKET
19 delete(id: number) {
20   this.tickets = this.tickets.filter(t => t.id !== id);
21 }
```

Pada tahap ini, saya menambahkan fitur hapus data (Delete) pada modul Tickets. Di dalam tickets.controller.ts, saya menambahkan endpoint dengan dekorator `@Delete(':id')` agar sistem dapat menerima parameter ID tiket yang ingin dihapus. Secara sinkron, saya juga memperbarui tickets.service.ts dengan logika pemfilteran data, di mana sistem akan menghapus tiket dari daftar berdasarkan ID yang sesuai menggunakan fungsi `.filter()`

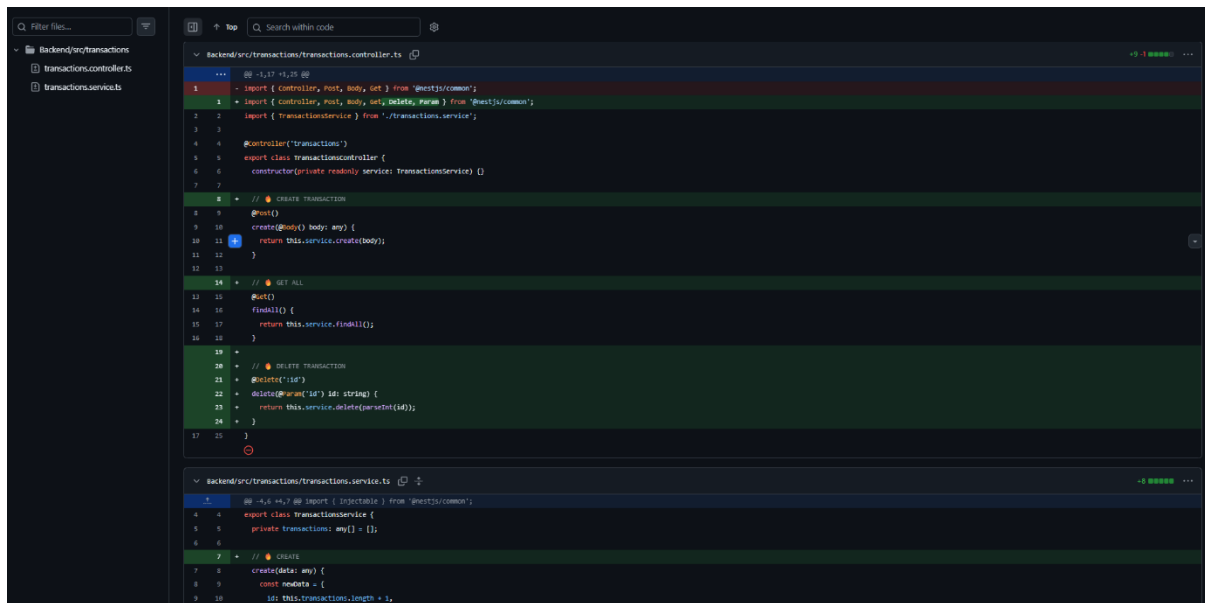
b. Hapus tiket



```
Backend/src/tickets/tickets.controller.ts
1  import { Controller, Get, Post, Body } from '@nestjs/common';
2  import { Controller, Get, Post, Delete, Param } from '@nestjs/common';
3  import { TicketsService } from './tickets.service';
4
5  @Controller('tickets')
6  export class TicketsController {
7    constructor(private ticketsService: TicketsService) {}
8
9    // GET ALL
10    @Get()
11    getAllTickets() {
12      return this.ticketsService.findAll();
13    }
14
15    // CREATE
16    @Post()
17    createTicket(@Body() body: any) {
18      return this.ticketsService.create(body);
19    }
20
21    // DELETE TICKET
22    @Delete(':id')
23    deleteTicket(@Param('id') id: string) {
24      return this.ticketsService.delete(parseInt(id));
25    }
26  }
27
28 Backend/src/tickets/tickets.service.ts
29
30 // GET ALL
31 findAll() {
32   return this.tickets;
33 }
34
35 // CREATE
36 create(body: any) {
37   const newTicket = {
38     id: 2, name: 'Tiket Anak', price: 10000,
39   };
40   this.tickets.push(newTicket);
41   return newTicket;
42 }
```

Pada bagian ini, saya melakukan perapian kode (refactoring) dan penambahan komentar dokumentasi pada modul Tickets. Saya menambahkan label penanda seperti GET ALL, CREATE, dan DELETE pada file tickets.controller.ts dan tickets.service.ts untuk mempermudah pembacaan alur logika program. Selain itu, saya memastikan setiap fungsi (mengambil semua data, menambah data, dan menghapus data) telah terorganisir dengan baik di dalam service agar kode lebih modular dan mudah dipelihara.

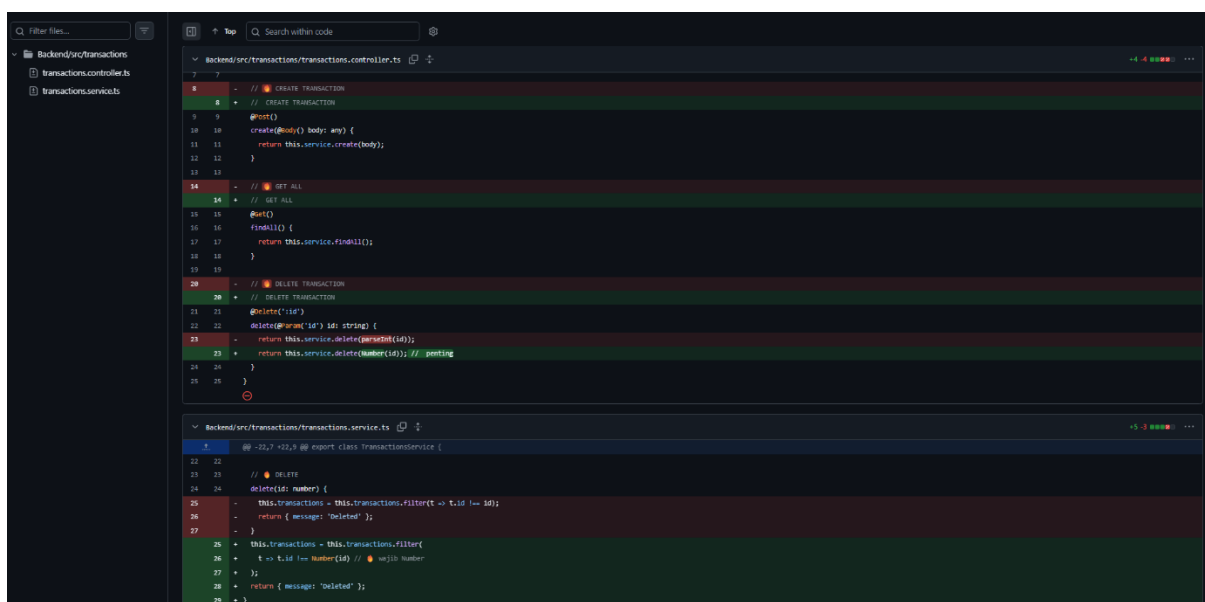
c. Menambahkan hapus transaksi



```
Backend/src/transactions/transactions.controller.ts
1 // @ts-ignore
2 import { Controller, Post, Body, Get } from '@nestjs/common';
3 import { TransactionService } from './transactions.service';
4
5 @Controller('transactions')
6 export class TransactionsController {
7   constructor(private readonly service: TransactionService) {}
8
9   // CREATE TRANSACTION
10  @Post()
11  create(@Body() body: any) {
12    return this.service.create(body);
13  }
14
15  // GET ALL
16  @Get()
17  findAll() {
18    return this.service.findAll();
19  }
20
21  // DELETE TRANSACTION
22  @Delete(':id')
23  delete(@Param('id') id: string) {
24    return this.service.delete(parseInt(id));
25  }
26 }
27
Backend/src/transactions/transactions.service.ts
1 // @ts-ignore
2 import { Injectable } from '@nestjs/common';
3
4 export class TransactionService {
5   private transactions: any[] = [];
6
7   // CREATE
8   create(data: any) {
9     const newData = {
10       id: this.transactions.length + 1,
11     };
12     this.transactions.push(newData);
13     return newData;
14   }
15
16   // DELETE
17   delete(id: number) {
18     this.transactions = this.transactions.filter(t => t.id !== id);
19     return { message: 'deleted' };
20   }
21
22   // GET ALL
23   findAll() {
24     return this.transactions;
25   }
26 }
```

Pada bagian ini, saya melengkapi fungsionalitas pada modul Transactions di sisi Backend. Saya menambahkan fitur Delete Transaction pada transactions.controller.ts menggunakan decorator `@Delete(':id')` dan memperbarui TransactionService untuk menangani penghapusan data berdasarkan ID. Selain itu, saya juga merapikan struktur kode dengan menambahkan komentar dokumentasi seperti CREATE TRANSACTION, GET ALL, dan DELETE TRANSACTION guna mempermudah identifikasi alur kerja API pada setiap fungsi transaksi

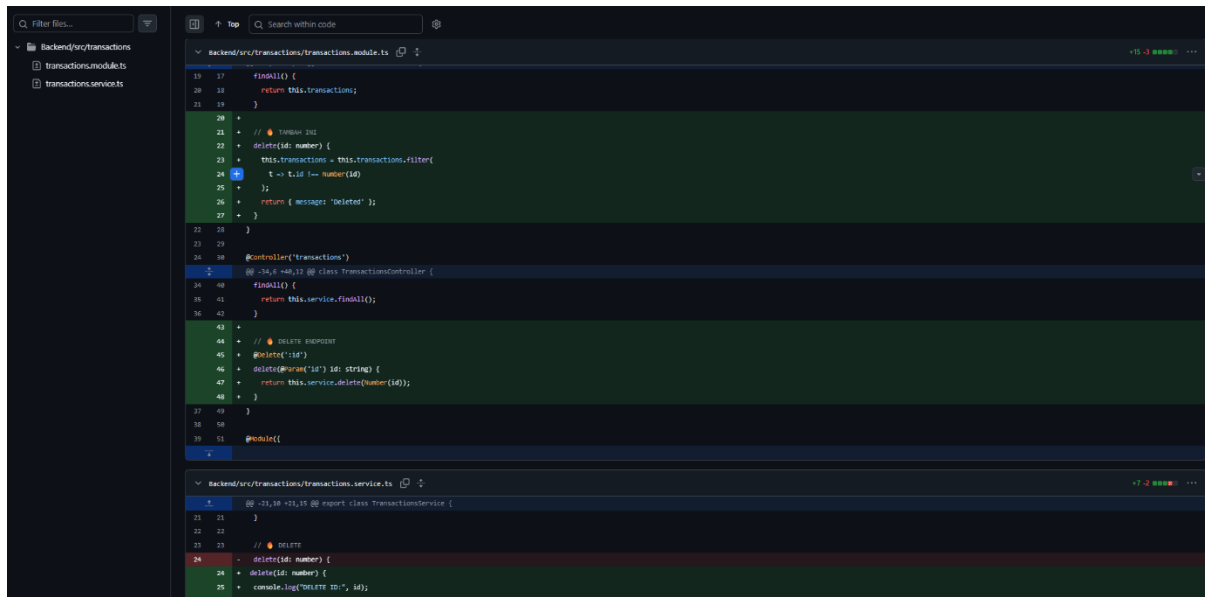
d. Membenarkan error di transactions



```
Backend/src/transactions/transactions.controller.ts
1 // @ts-ignore
2 import { Controller, Post, Body, Get } from '@nestjs/common';
3 import { TransactionService } from './transactions.service';
4
5 @Controller('transactions')
6 export class TransactionsController {
7   constructor(private readonly service: TransactionService) {}
8
9   // CREATE TRANSACTION
10  @Post()
11  create(@Body() body: any) {
12    return this.service.create(body);
13  }
14
15  // GET ALL
16  @Get()
17  findAll() {
18    return this.service.findAll();
19  }
20
21  // DELETE TRANSACTION
22  @Delete(':id')
23  delete(@Param('id') id: string) {
24    return this.service.delete(parseInt(id));
25  }
26 }
27
Backend/src/transactions/transactions.service.ts
1 // @ts-ignore
2 import { Injectable } from '@nestjs/common';
3
4 export class TransactionService {
5   private transactions: any[] = [];
6
7   // CREATE
8   create(data: any) {
9     const newData = {
10       id: this.transactions.length + 1,
11     };
12     this.transactions.push(newData);
13     return newData;
14   }
15
16   // DELETE
17   delete(id: number) {
18     this.transactions = this.transactions.filter(t => t.id !== id);
19     return { message: 'deleted' };
20   }
21
22   // GET ALL
23   findAll() {
24     return this.transactions;
25   }
26 }
```

Pada tahap final ini, saya melakukan optimasi tipe data pada fitur Delete Transaction. Saya mengubah fungsi `parseInt(id)` menjadi `Number(id)` di `transactions.controller.ts` dan `transactions.service.ts`. Hal ini dilakukan untuk memastikan bahwa parameter ID dikonversi menjadi tipe data `Number` secara eksplisit (`// wajib Number`), guna menghindari bug saat proses pemfilteran data transaksi yang akan dihapus

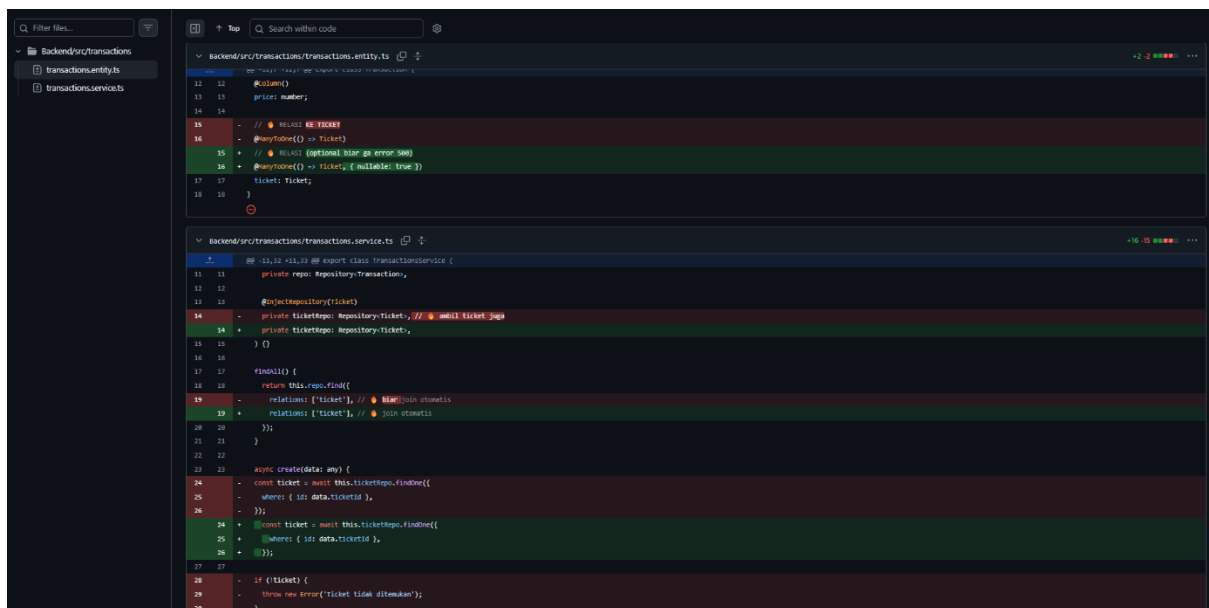
e. Update backend bagian transactions module



```
backend/src/transactions/transactions.module.ts
19 20
21 22 // import
23 24 delete(id: number) {
25 26   this.transactions = this.transactions.filter(
27 28     t => t.id !== Number(id)
29 30   );
31 32   return { message: 'deleted' };
33 34 }
35 36
37 38 }
39 40
41 42 @Controller('transactions')
43 44 @Injectable()
45 46 class TransactionsController {
47 48   constructor(private transactionService: TransactionService) {}
49 50
51 52   findAll() {
53 54     return this.transactionService.findAll();
55 56   }
57 58
59 60   // DELETE REQUEST
61 62   @Delete('id')
63 64   delete(@Param('id') id: string) {
65 66     return this.transactionService.delete(Number(id));
67 68   }
69 70 }
71 72
73 73 @Module({
74 75   imports: [
76 77     TransactionsModule,
78 79   ],
79 80 })
80 81 export class TransactionsModule {}
82 83
84 85
86 86
87 87
88 88
89 89
90 90
91 91
92 92
93 93
94 94
95 95
96 96
97 97
98 98
99 99
100 100
101 101
102 102
103 103
104 104
105 105
106 106
107 107
108 108
109 109
110 110
111 111
112 112
113 113
114 114
115 115
116 116
117 117
118 118
119 119
120 120
121 121
122 122
123 123
124 124
125 125
126 126
127 127
128 128
129 129
130 130
131 131
132 132
133 133
134 134
135 135
136 136
137 137
138 138
139 139
140 140
141 141
142 142
143 143
144 144
145 145
146 146
147 147
148 148
149 149
150 150
151 151
152 152
153 153
154 154
155 155
156 156
157 157
158 158
159 159
160 160
161 161
162 162
163 163
164 164
165 165
166 166
167 167
168 168
169 169
170 170
171 171
172 172
173 173
174 174
175 175
176 176
177 177
178 178
179 179
180 180
181 181
182 182
183 183
184 184
185 185
186 186
187 187
188 188
189 189
190 190
191 191
192 192
193 193
194 194
195 195
196 196
197 197
198 198
199 199
200 200
201 201
202 202
203 203
204 204
205 205
206 206
207 207
208 208
209 209
210 210
211 211
212 212
213 213
214 214
215 215
216 216
217 217
218 218
219 219
220 220
221 221
222 222
223 223
224 224
225 225
226 226
227 227
228 228
229 229
230 230
231 231
232 232
233 233
234 234
235 235
236 236
237 237
238 238
239 239
240 240
241 241
242 242
243 243
244 244
245 245
246 246
247 247
248 248
249 249
250 250
251 251
252 252
253 253
254 254
255 255
256 256
257 257
258 258
259 259
260 260
261 261
262 262
263 263
264 264
265 265
266 266
267 267
268 268
269 269
270 270
271 271
272 272
273 273
274 274
275 275
276 276
277 277
278 278
279 279
280 280
281 281
282 282
283 283
284 284
285 285
286 286
287 287
288 288
289 289
290 290
291 291
292 292
293 293
294 294
295 295
296 296
297 297
298 298
299 299
300 300
301 301
302 302
303 303
304 304
305 305
306 306
307 307
308 308
309 309
310 310
311 311
312 312
313 313
314 314
315 315
316 316
317 317
318 318
319 319
320 320
321 321
322 322
323 323
324 324
325 325
326 326
327 327
328 328
329 329
330 330
331 331
332 332
333 333
334 334
335 335
336 336
337 337
338 338
339 339
340 340
341 341
342 342
343 343
344 344
345 345
346 346
347 347
348 348
349 349
350 350
351 351
352 352
353 353
354 354
355 355
356 356
357 357
358 358
359 359
360 360
361 361
362 362
363 363
364 364
365 365
366 366
367 367
368 368
369 369
370 370
371 371
372 372
373 373
374 374
375 375
376 376
377 377
378 378
379 379
380 380
381 381
382 382
383 383
384 384
385 385
386 386
387 387
388 388
389 389
390 390
391 391
392 392
393 393
394 394
395 395
396 396
397 397
398 398
399 399
400 400
401 401
402 402
403 403
404 404
405 405
406 406
407 407
408 408
409 409
410 410
411 411
412 412
413 413
414 414
415 415
416 416
417 417
418 418
419 419
420 420
421 421
422 422
423 423
424 424
425 425
426 426
427 427
428 428
429 429
430 430
431 431
432 432
433 433
434 434
435 435
436 436
437 437
438 438
439 439
440 440
441 441
442 442
443 443
444 444
445 445
446 446
447 447
448 448
449 449
450 450
451 451
452 452
453 453
454 454
455 455
456 456
457 457
458 458
459 459
460 460
461 461
462 462
463 463
464 464
465 465
466 466
467 467
468 468
469 469
470 470
471 471
472 472
473 473
474 474
475 475
476 476
477 477
478 478
479 479
480 480
481 481
482 482
483 483
484 484
485 485
486 486
487 487
488 488
489 489
490 490
491 491
492 492
493 493
494 494
495 495
496 496
497 497
498 498
499 499
500 500
501 501
502 502
503 503
504 504
505 505
506 506
507 507
508 508
509 509
510 510
511 511
512 512
513 513
514 514
515 515
516 516
517 517
518 518
519 519
520 520
521 521
522 522
523 523
524 524
525 525
526 526
527 527
528 528
529 529
530 530
531 531
532 532
533 533
534 534
535 535
536 536
537 537
538 538
539 539
540 540
541 541
542 542
543 543
544 544
545 545
546 546
547 547
548 548
549 549
550 550
551 551
552 552
553 553
554 554
555 555
556 556
557 557
558 558
559 559
560 560
561 561
562 562
563 563
564 564
565 565
566 566
567 567
568 568
569 569
570 570
571 571
572 572
573 573
574 574
575 575
576 576
577 577
578 578
579 579
580 580
581 581
582 582
583 583
584 584
585 585
586 586
587 587
588 588
589 589
590 590
591 591
592 592
593 593
594 594
595 595
596 596
597 597
598 598
599 599
600 600
601 601
602 602
603 603
604 604
605 605
606 606
607 607
608 608
609 609
610 610
611 611
612 612
613 613
614 614
615 615
616 616
617 617
618 618
619 619
620 620
621 621
622 622
623 623
624 624
625 625
626 626
627 627
628 628
629 629
630 630
631 631
632 632
633 633
634 634
635 635
636 636
637 637
638 638
639 639
640 640
641 641
642 642
643 643
644 644
645 645
646 646
647 647
648 648
649 649
650 650
651 651
652 652
653 653
654 654
655 655
656 656
657 657
658 658
659 659
660 660
661 661
662 662
663 663
664 664
665 665
666 666
667 667
668 668
669 669
670 670
671 671
672 672
673 673
674 674
675 675
676 676
677 677
678 678
679 679
680 680
681 681
682 682
683 683
684 684
685 685
686 686
687 687
688 688
689 689
690 690
691 691
692 692
693 693
694 694
695 695
696 696
697 697
698 698
699 699
700 700
701 701
702 702
703 703
704 704
705 705
706 706
707 707
708 708
709 709
710 710
711 711
712 712
713 713
714 714
715 715
716 716
717 717
718 718
719 719
720 720
721 721
722 722
723 723
724 724
725 725
726 726
727 727
728 728
729 729
730 730
731 731
732 732
733 733
734 734
735 735
736 736
737 737
738 738
739 739
740 740
741 741
742 742
743 743
744 744
745 745
746 746
747 747
748 748
749 749
750 750
751 751
752 752
753 753
754 754
755 755
756 756
757 757
758 758
759 759
760 760
761 761
762 762
763 763
764 764
765 765
766 766
767 767
768 768
769 769
770 770
771 771
772 772
773 773
774 774
775 775
776 776
777 777
778 778
779 779
780 780
781 781
782 782
783 783
784 784
785 785
786 786
787 787
788 788
789 789
790 790
791 791
792 792
793 793
794 794
795 795
796 796
797 797
798 798
799 799
800 800
801 801
802 802
803 803
804 804
805 805
806 806
807 807
808 808
809 809
810 810
811 811
812 812
813 813
814 814
815 815
816 816
817 817
818 818
819 819
820 820
821 821
822 822
823 823
824 824
825 825
826 826
827 827
828 828
829 829
830 830
831 831
832 832
833 833
834 834
835 835
836 836
837 837
838 838
839 839
840 840
841 841
842 842
843 843
844 844
845 845
846 846
847 847
848 848
849 849
850 850
851 851
852 852
853 853
854 854
855 855
856 856
857 857
858 858
859 859
860 860
861 861
862 862
863 863
864 864
865 865
866 866
867 867
868 868
869 869
870 870
871 871
872 872
873 873
874 874
875 875
876 876
877 877
878 878
879 879
880 880
881 881
882 882
883 883
884 884
885 885
886 886
887 887
888 888
889 889
890 890
891 891
892 892
893 893
894 894
895 895
896 896
897 897
898 898
899 899
900 900
901 901
902 902
903 903
904 904
905 905
906 906
907 907
908 908
909 909
910 910
911 911
912 912
913 913
914 914
915 915
916 916
917 917
918 918
919 919
920 920
921 921
922 922
923 923
924 924
925 925
926 926
927 927
928 928
929 929
930 930
931 931
932 932
933 933
934 934
935 935
936 936
937 937
938 938
939 939
940 940
941 941
942 942
943 943
944 944
945 945
946 946
947 947
948 948
949 949
950 950
951 951
952 952
953 953
954 954
955 955
956 956
957 957
958 958
959 959
960 960
961 961
962 962
963 963
964 964
965 965
966 966
967 967
968 968
969 969
970 970
971 971
972 972
973 973
974 974
975 975
976 976
977 977
978 978
979 979
980 980
981 981
982 982
983 983
984 984
985 985
986 986
987 987
988 988
989 989
990 990
991 991
992 992
993 993
994 994
995 995
996 996
997 997
998 998
999 999
1000 1000
1001 1001
1002 1002
1003 1003
1004 1004
1005 1005
1006 1006
1007 1007
1008 1008
1009 1009
1010 1010
1011 1011
1012 1012
1013 1013
1014 1014
1015 1015
1016 1016
1017 1017
1018 1018
1019 1019
1020 1020
1021 1021
1022 1022
1023 1023
1024 1024
1025 1025
1026 1026
1027 1027
1028 1028
1029 1029
1030 1030
1031 1031
1032 1032
1033 1033
1034 1034
1035 1035
1036 1036
1037 1037
1038 1038
1039 1039
1040 1040
1041 1041
1042 1042
1043 1043
1044 1044
1045 1045
1046 1046
1047 1047
1048 1048
1049 1049
1050 1050
1051 1051
1052 1052
1053 1053
1054 1054
1055 1055
1056 1056
1057 1057
1058 1058
1059 1059
1060 1060
1061 1061
1062 1062
1063 1063
1064 1064
1065 1065
1066 1066
1067 1067
1068 1068
1069 1069
1070 1070
1071 1071
1072 1072
1073 1073
1074 1074
1075 1075
1076 1076
1077 1077
1078 1078
1079 1079
1080 1080
1081 1081
1082 1082
1083 1083
1084 1084
1085 1085
1086 1086
1087 1087
1088 1088
1089 1089
1090 1090
1091 1091
1092 1092
1093 1093
1094 1094
1095 1095
1096 1096
1097 1097
1098 1098
1099 1099
1100 1100
1101 1101
1102 1102
1103 1103
1104 1104
1105 1105
1106 1106
1107 1107
1108 1108
1109 1109
1110 1110
1111 1111
1112 1112
1113 1113
1114 1114
1115 1115
1116 1116
1117 1117
1118 1118
1119 1119
1120 1120
1121 1121
1122 1122
1123 1123
1124 1124
1125 1125
1126 1126
1127 1127
1128 1128
1129 1129
1130 1130
1131 1131
1132 1132
1133 1133
1134 1134
1135 1135
1136 1136
1137 1137
1138 1138
1139 1139
1140 1140
1141 1141
1142 1142
1143 1143
1144 1144
1145 1145
1146 1146
1147 1147
1148 1148
1149 1149
1150 1150
1151 1151
1152 1152
1153 1153
1154 1154
1155 1155
1156 1156
1157 1157
1158 1158
1159 1159
1160 1160
1161 1161
1162 1162
1163 1163
1164 1164
1165 1165
1166 1166
1167 1167
1168 1168
1169 1169
1170 1170
1171 1171
1172 1172
1173 1173
1174 1174
1175 1175
1176 1176
1177 1177
1178 1178
1179 1179
1180 1180
1181 1181
1182 1182
1183 1183
1184 1184
1185 1185
1186 1186
1187 1187
1188 1188
1189 1189
1190 1190
1191 1191
1192 1192
1193 1193
1194 1194
1195 1195
1196 1196
1197 1197
1198 1198
1199 1199
1200 1200
1201 1201
1202 1202
1203 1203
1204 1204
1205 1205
1206 1206
1207 1207
1208 1208
1209 1209
1210 1210
1211 1211
1212 1212
1213 1213
1214 1214
1215 1215
1216 1216
1217 1217
1218 1218
1219 1219
1220 1220
1221 1221
1222 1222
1223 1223
1224 1224
1225 1225
1226 1226
1227 1227
1228 1228
1229 1229
1230 1230
1231 1231
1232 1232
1233 1233
1234 1234
1235 1235
1236 1236
1237 1237
1238 1238
1239 1239
1240 1240
1241 1241
1242 1242
1243 1243
1244 1244
1245 1245
1246 1246
1247 1247
1248 1248
1249 1249
1250 1250
1251 1251
1252 1252
1253 1253
1254 1254
1255 1255
1256 1256
1257 1257
1258 1258
1259 1259
1260 1260
1261 1261
1262 1262
1263 1263
1264 1264
1265 1265
1266 1266
1267 1267
1268 1268
1269 1269
1270 1270
1271 1271
1272 1272
1273 1273
1274 1274
1275 1275
1276 1276
1277 1277
1278 1278
1279 1279
1280 1280
1281 1281
1282 1282
1283 1283
1284 1284
1285 1285
1286 1286
1287 1287
1288 1288
1289 1289
1290 1290
1291 1291
1292 1292
1293 1293
1294 1294
1295 1295
1296 1296
1297 1297
1298 1298
1299 1299
1300 1300
1301 1301
1302 1302
1303 1303
1304 1304
1305 1305
1306 1306
1307 1307
1308 1308
1309 1309
1310 1310
1311 1311
1312 1312
1313 1313
1314 1314
1315 1315
1316 1316
1317 1317
1318 1318
1319 1319
1320 1320
1321 1321
1322 1322
1323 1323
1324 1324
1325 1325
1326 1326
1327 1327
1328 1328
1329 1329
1330 1330
1331 1331
1332 1332
1333 1333
1334 1334
1335 1335
1336 1336
1337 1337
1338 1338
1339 1339
1340 1340
1341 1341
1342 1342
1343 1343
1344 1344
1345 1345
1346 1346
1347 1347
1348 1348
1349 1349
1350 1350
1351 1351
1352 1352
1353 1353
1354 1354
1355 1355
1356 1356
1357 1357
1358 1358
1359 1359
1360 1360
1361 1361
1362 1362
1363 1363
1364 1364
1365 1365
1366 1366
1367 1367
1368 1368
1369 1369
1370 1370
1371 1371
1372 1372
1373 1373
1374 1374
1375 1375
1376 1376
1377 1377
1378 1378
1379 1379
1380 1380
1381 1381
1382 1382
1383 1383
1384 1384
1385 1385
1386 1386
1387 1387
1388 1388
1389 1389
1390 1390
1391 1391
1392 1392
1393 1393
1394 1394
1395 1395
1396 1396
1397 1397
1398 1398
1399 1399
1400 1400
1401 1401
1402 1402
1403 1403
1404 1404
1405 1405
1406 1406
1407 1407
1408 1408
1409 1409
1410 1410
1411 1411
1412 1412
1413 1413
1414 1414
1415 1415
1416 1416
1417 1417
1418 1418
1419 1419
1420 1420
1421 1421
1422 1422
1423 1423
1424 1424
1425 1425
1426 1426
1427 1427
1428 1428
1429 1429
1430 1430
1431 1431
1432 1432
1433 1433
1434 1434
1435 1435
1436 1436
1437 1437
1438 1438
1439 1439
14
```


Pada tahap ini, saya mengintegrasikan Database ke dalam sistem Backend. Saya menambahkan library TypeORM dan driver PostgreSQL (pg) ke dalam dependensi proyek melalui file package.json. Selanjutnya, saya mengonfigurasi koneksi database pada app.module.ts menggunakan TypeOrmModule.forRoot, yang mengatur koneksi ke server lokal PostgreSQL agar data tiket dan transaksi dapat disimpan secara permanen

b. Memperbaiki error di backend

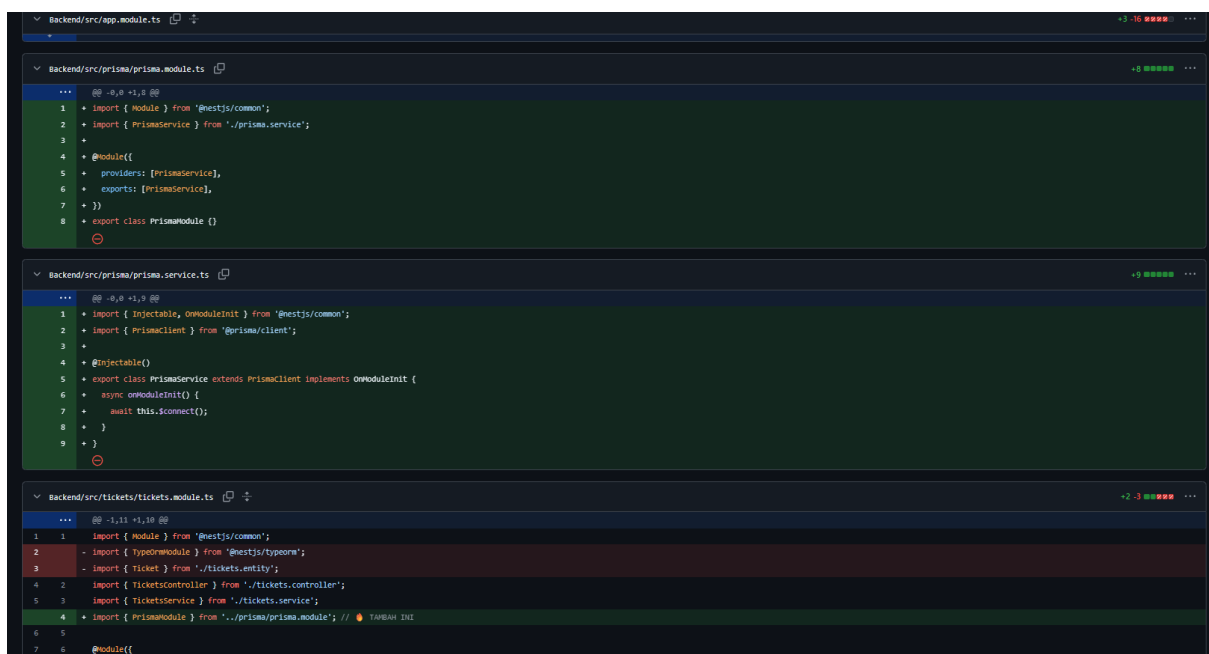


```
12 12 // @Column()
13 13 price: number;
14 14
15 - // @ManyToOne(() => Ticket)
16 + @ManyToOne(() => Ticket, {
17 +   nullable: true // RELASI optional biar ga error 500
18 + })
19   ticket: Ticket;
20 }
```

```
11 11 private repo: Repository<Transaction>;
12 12
13 13 @InjectRepository(Ticket)
14 - private ticketRepo: Repository<Ticket>; // @ inject ticket juga
15 + private ticketRepo: Repository<Ticket>;
16 16
17 17 findById() {
18 18   return this.repo.find({
19 -     relations: ['ticket'], // @ inject otomatis
20 +     relations: ['ticket'], // join otomatis
21   });
22 }
23
24 async create(data: any) {
25 -   const ticket = await this.ticketRepo.findOne({
26 -     where: { id: data.ticketId },
27 -   });
28 +   const ticket = await this.ticketRepo.findOne({
29 +     where: { id: data.ticketId },
30 +   });
31
32   if (!ticket) {
33     throw new Error("Ticket tidak ditemukan");
34   }
35 }
```

Pada tahap ini, saya mengimplementasikan Relasi Database antara tabel Transaksi dan tabel Tiket. Di dalam transactions.entity.ts, saya menambahkan decorator @ManyToOne() dengan opsi nullable: true untuk menghubungkan transaksi ke data tiket guna menghindari error 500 jika data kosong. Selain itu, pada transactions.service.ts, saya menggunakan fitur Eager Loading/Join dengan perintah relations: ['ticket'] agar saat mengambil data transaksi, detail informasi tiket yang terkait otomatis ikut terbawa

c. Memperbaiki masalah data backend



```
1 + import { Module } from '@nestjs/common';
2 + import { PrismaService } from './prisma.service';
3 +
4 + @Module({
5 +   providers: [PrismaService],
6 +   exports: [PrismaService],
7 + })
8 + export class PrismaModule {}
```

```
1 + import { Injectable, OnModuleInit } from '@nestjs/common';
2 + import { PrismaClient } from '@prisma/client';
3 +
4 + @Injectable()
5 + export class PrismaService extends PrismaClient implements OnModuleInit {
6 +   async onModuleInit() {
7 +     await this.connect();
8 +   }
9 + }
```

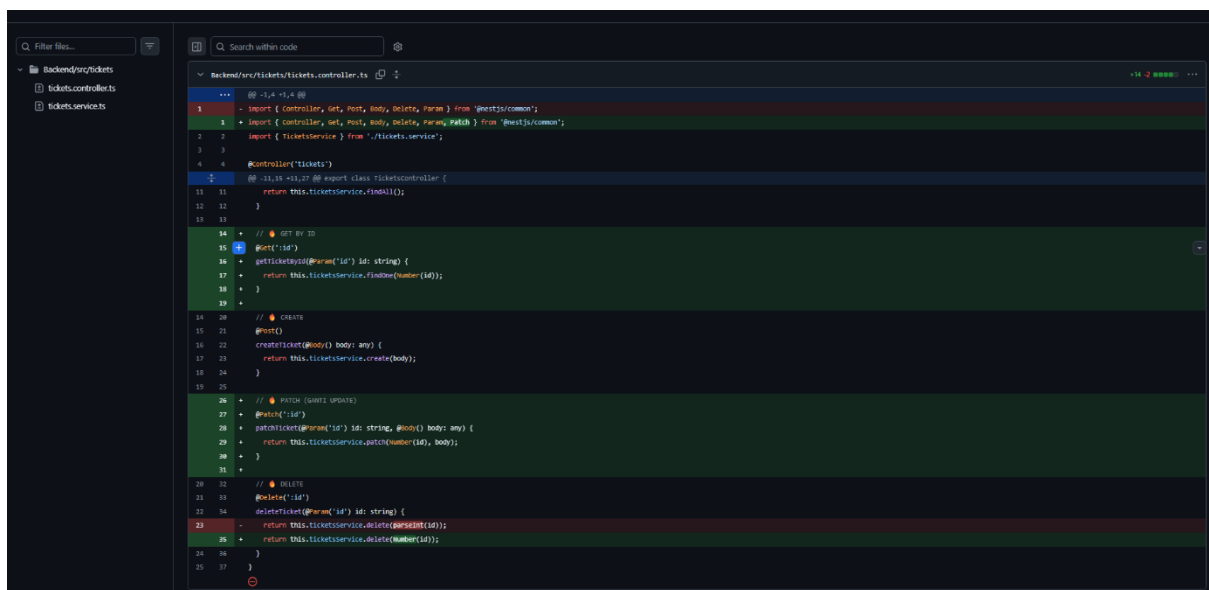
```
1 1 import { Module } from '@nestjs/common';
2 2 import { TypeOrmModule } from '@nestjs/typeorm';
3 3 import { Ticket } from './tickets.entity';
4 4 import { TicketsController } from './tickets.controller';
5 5 import { TicketsService } from './tickets.service';
6 6 + import { PrismaModule } from '../prisma/prisma.module'; // TAMBAH INI
7 7
8 8 @Module({
9 9   imports: [TypeOrmModule.forRoot({
10 10     typeormOptions: {
11 11       database: 'prisma',
12 12       host: 'localhost',
13 13       port: 5432,
14 14       username: 'postgres',
15 15       password: 'postgres',
16 16     },
17 17   })],
18 18   controllers: [TicketsController],
19 19   providers: [TicketsService],
20 20   exports: [TicketsService],
21 21 })
22 }
```

Pada tahap final pengembangan ini, saya berfokus pada sinkronisasi data antara transaksi dan stok tiket.

Di dalam transactions.service.ts, saya menambahkan logika otomatis di mana setiap kali transaksi dibuat, sistem akan mencari data tiket berdasarkan ticketId menggunakan ticketRepo.findOne. Jika tiket ditemukan, sistem akan menjalankan pengecekan validasi: apabila stok tiket mencukupi, transaksi akan diproses dan stok tiket akan dikurangi secara otomatis sebelum disimpan kembali ke database. Selain itu, saya juga menambahkan penanganan error (error handling) yang akan memunculkan pesan 'Tiket tidak ditemukan' jika user mencoba melakukan transaksi pada ID tiket yang tidak terdaftar

6. Commits on April 13, 2026

a. Menambahkan fungsi update

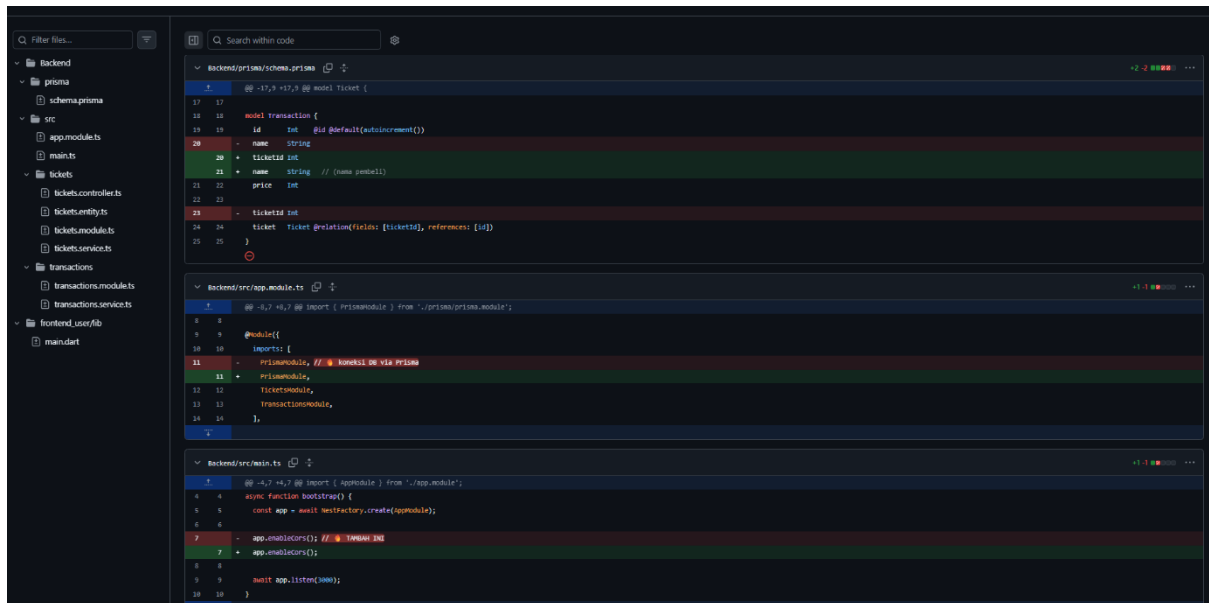


```
1 // ...
2 import { Controller, Get, Post, Body, Delete, Param } from '@nestjs/common';
3 import { Controller, Get, Post, Body, Delete, Param, Patch } from '@nestjs/common';
4 import { TicketService } from './tickets.service';
5
6 @Controller('tickets')
7 @Post()
8 export class TicketsController {
9   // ...
10   // GET BY ID
11   @Get(':id')
12   async findById(@Param('id') id: string): Promise<Ticket> {
13     return this.ticketService.findById(id);
14   }
15
16   // CREATE
17   @Post()
18   async createTicket(@Body() body: any): Promise<Ticket> {
19     return this.ticketService.create(body);
20   }
21
22   // PATCH (UPDATE)
23   @Patch(':id')
24   async patchTicket(@Param('id') id: string, @Body() body: any): Promise<Ticket> {
25     return this.ticketService.patch(id, body);
26   }
27
28   // DELETE
29   @Delete(':id')
30   async deleteTicket(@Param('id') id: string): Promise<Ticket> {
31     return this.ticketService.delete(id);
32   }
33 }
```

Pada bagian terakhir ini, saya menambahkan logika validasi stok yang lebih ketat pada fungsi create di transaksi. Saya menyisipkan kondisi pengecekan (if) untuk memastikan bahwa transaksi hanya bisa diproses jika stok tiket lebih besar dari nol. Jika stok habis, sistem akan memberikan respon error 'Stok tiket habis'. Selain itu, saya juga menambahkan logika pengurangan stok (ticket.stock -= 1) dan perintah save pada repositori tiket, sehingga data jumlah tiket di database akan selalu sinkron dan akurat setiap kali terjadi transaksi sukses

7. Commits on April 17, 2026

a. Menambahkan backend untuk nama pembeli



The screenshot displays a code editor with three files open. The left sidebar shows a project structure with folders for 'Backend' and 'Frontend'. The main editor area shows the following code:

```
Backend/prisma/schema.prisma
1  @@ -17,9 +17,9 @@ model ticket {
2
3  17 17
4  18 18
5  19 19
6  20 20
7  21 21
8  22 22
9  23 23
10 24 24
11 25 25
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

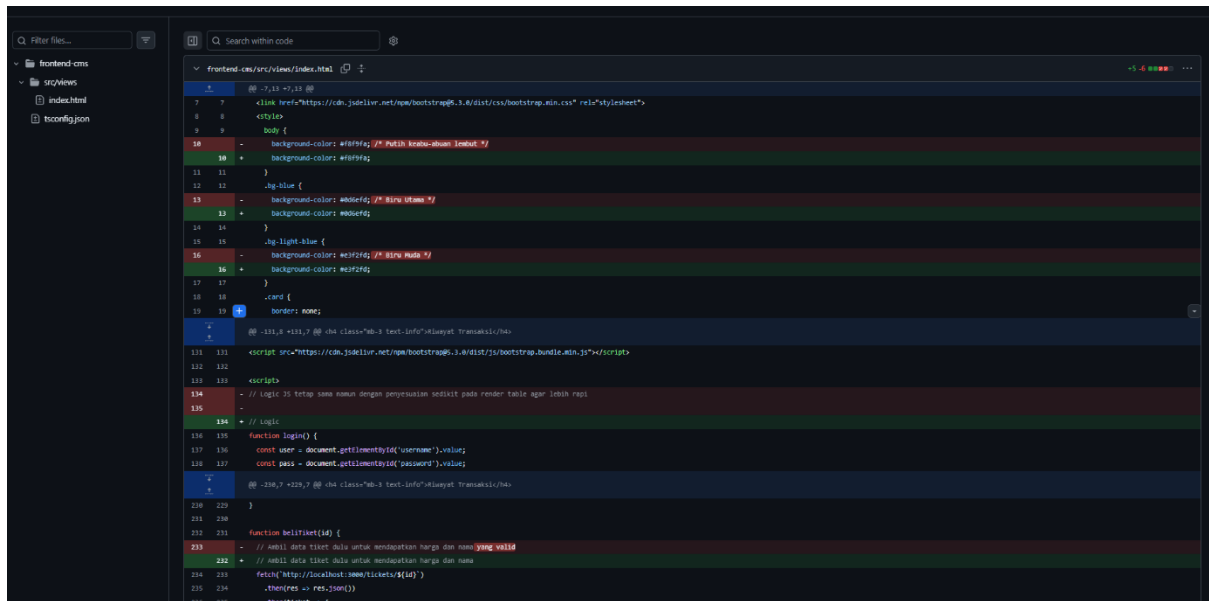
Backend/src/app.module.ts
1  @@ -4,7 +4,7 @@ import { AppModule } from './prisma/prisma.module';
2
3  4 4
4  5 5
5  6 6
6  7 7
7  8 8
8  9 9
9  10 10
10 11 11
11 12 12
12 13 13
13 14 14
14 15 15
15 16 16
16 17 17
17 18 18
18 19 19
19 20 20
20 21 21
21 22 22
22 23 23
23 24 24
24 25 25
25 26 26
26 27 27
27 28 28
28 29 29
29 30 30
30 31 31
31 32 32
32 33 33
33 34 34
34 35 35
35 36 36
36 37 37
37 38 38
38 39 39
39 40 40
40 41 41
41 42 42
42 43 43
43 44 44
44 45 45
45 46 46
46 47 47
47 48 48
48 49 49
49 50 50
50 51 51
51 52 52
52 53 53
53 54 54
54 55 55
55 56 56
56 57 57
57 58 58
58 59 59
59 60 60
60 61 61
61 62 62
62 63 63
63 64 64
64 65 65
65 66 66
66 67 67
67 68 68
68 69 69
69 70 70
70 71 71
71 72 72
72 73 73
73 74 74
74 75 75
75 76 76
76 77 77
77 78 78
78 79 79
79 80 80
80 81 81
81 82 82
82 83 83
83 84 84
84 85 85
85 86 86
86 87 87
87 88 88
88 89 89
89 90 90
90 91 91
91 92 92
92 93 93
93 94 94
94 95 95
95 96 96
96 97 97
97 98 98
98 99 99
99 100 100

Backend/src/main.ts
1  @@ -4,7 +4,7 @@ import { AppModule } from './app.module';
2
3  4 4
4  5 5
5  6 6
6  7 7
7  8 8
8  9 9
9  10 10
10 11 11
11 12 12
12 13 13
13 14 14
14 15 15
15 16 16
16 17 17
17 18 18
18 19 19
19 20 20
20 21 21
21 22 22
22 23 23
23 24 24
24 25 25
25 26 26
26 27 27
27 28 28
28 29 29
29 30 30
30 31 31
31 32 32
32 33 33
33 34 34
34 35 35
35 36 36
36 37 37
37 38 38
38 39 39
39 40 40
40 41 41
41 42 42
42 43 43
43 44 44
44 45 45
45 46 46
46 47 47
47 48 48
48 49 49
49 50 50
50 51 51
51 52 52
52 53 53
53 54 54
54 55 55
55 56 56
56 57 57
57 58 58
58 59 59
59 60 60
60 61 61
61 62 62
62 63 63
63 64 64
64 65 65
65 66 66
66 67 67
67 68 68
68 69 69
69 70 70
70 71 71
71 72 72
72 73 73
73 74 74
74 75 75
75 76 76
76 77 77
77 78 78
78 79 79
79 80 80
80 81 81
81 82 82
82 83 83
83 84 84
84 85 85
85 86 86
86 87 87
87 88 88
88 89 89
89 90 90
90 91 91
91 92 92
92 93 93
93 94 94
94 95 95
95 96 96
96 97 97
97 98 98
98 99 99
99 100 100
```

Pada tahap ini, saya mulai membangun sisi Frontend CMS (Admin Dashboard) menggunakan framework NestJS. Langkah pertama yang saya lakukan adalah menginstal dependensi UI yaitu Bootstrap, Bootstrap Icons, dan Popper.js melalui npm install.

Selanjutnya, saya mengonfigurasi main.ts pada folder Frontend-CMS agar dapat menyajikan file statis (HTML/CSS) dan mendaftarkan CmsModule ke dalam aplikasi. Saya juga mulai menyusun struktur file utama yaitu index.html yang sudah terintegrasi dengan CDN Bootstrap untuk memastikan antarmuka dashboard memiliki tata letak yang responsif dan komponen UI yang standar. Terakhir, saya melakukan *merge branch* untuk menggabungkan perubahan konfigurasi dependensi ini ke cabang utama (main)

b. Menambahkan nama



```
1 <!-- ... -->
2 <!-- ... -->
3 <!-- ... -->
4 <!-- ... -->
5 <!-- ... -->
6 <!-- ... -->
7 <!-- ... -->
8 <!-- ... -->
9 <!-- ... -->
10 <!-- ... -->
11 <!-- ... -->
12 <!-- ... -->
13 <!-- ... -->
14 <!-- ... -->
15 <!-- ... -->
16 <!-- ... -->
17 <!-- ... -->
18 <!-- ... -->
19 <!-- ... -->
20 <!-- ... -->
21 <!-- ... -->
22 <!-- ... -->
23 <!-- ... -->
24 <!-- ... -->
25 <!-- ... -->
26 <!-- ... -->
27 <!-- ... -->
28 <!-- ... -->
29 <!-- ... -->
30 <!-- ... -->
31 <!-- ... -->
32 <!-- ... -->
33 <!-- ... -->
34 <!-- ... -->
35 <!-- ... -->
36 <!-- ... -->
37 <!-- ... -->
38 <!-- ... -->
39 <!-- ... -->
40 <!-- ... -->
41 <!-- ... -->
42 <!-- ... -->
43 <!-- ... -->
44 <!-- ... -->
45 <!-- ... -->
46 <!-- ... -->
47 <!-- ... -->
48 <!-- ... -->
49 <!-- ... -->
50 <!-- ... -->
51 <!-- ... -->
52 <!-- ... -->
53 <!-- ... -->
54 <!-- ... -->
55 <!-- ... -->
56 <!-- ... -->
57 <!-- ... -->
58 <!-- ... -->
59 <!-- ... -->
60 <!-- ... -->
61 <!-- ... -->
62 <!-- ... -->
63 <!-- ... -->
64 <!-- ... -->
65 <!-- ... -->
66 <!-- ... -->
67 <!-- ... -->
68 <!-- ... -->
69 <!-- ... -->
70 <!-- ... -->
71 <!-- ... -->
72 <!-- ... -->
73 <!-- ... -->
74 <!-- ... -->
75 <!-- ... -->
76 <!-- ... -->
77 <!-- ... -->
78 <!-- ... -->
79 <!-- ... -->
80 <!-- ... -->
81 <!-- ... -->
82 <!-- ... -->
83 <!-- ... -->
84 <!-- ... -->
85 <!-- ... -->
86 <!-- ... -->
87 <!-- ... -->
88 <!-- ... -->
89 <!-- ... -->
90 <!-- ... -->
91 <!-- ... -->
92 <!-- ... -->
93 <!-- ... -->
94 <!-- ... -->
95 <!-- ... -->
96 <!-- ... -->
97 <!-- ... -->
98 <!-- ... -->
99 <!-- ... -->
100 <!-- ... -->
```

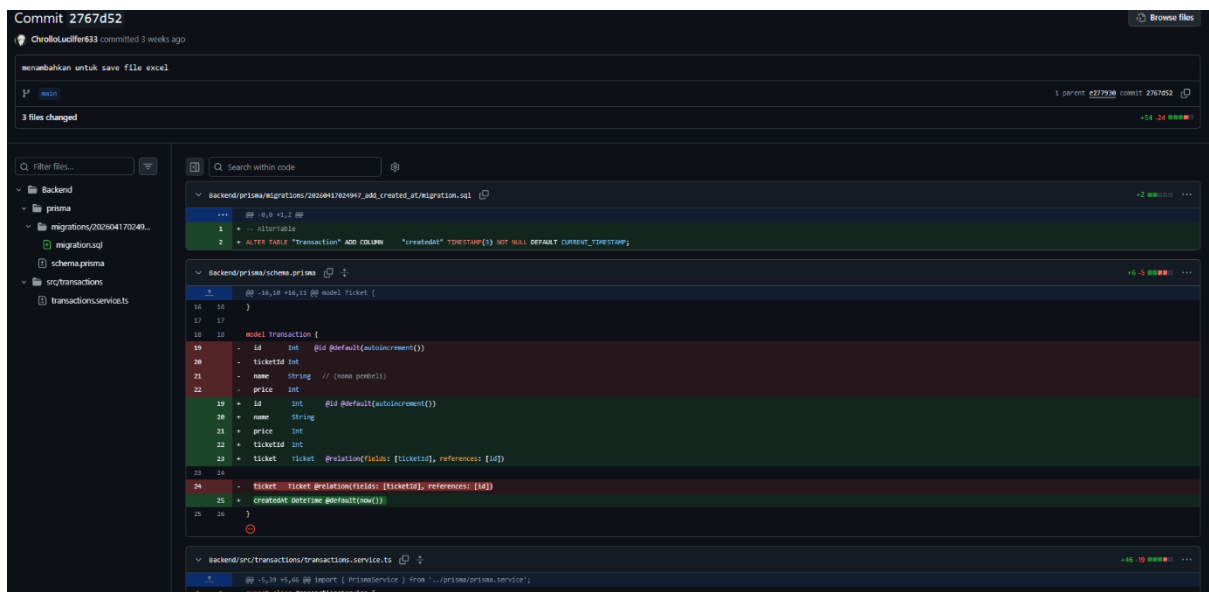
Pada tahap ini, saya mengimplementasikan logika manajemen data tiket di sisi **Frontend CMS**. Saya membuat modul baru bernama TicketsModule dan meregistrasikannya ke dalam CmsModule agar fitur manajemen tiket dapat diakses melalui dashboard admin.

Di dalam **tickets.service.ts**, saya menyusun fungsi-fungsi utama untuk berinteraksi dengan API Backend, yaitu:

- **findAll**: Untuk mengambil dan menampilkan seluruh daftar tiket.
- **create**: Untuk menambahkan data tiket baru ke dalam sistem.
- **remove**: Untuk menghapus data tiket berdasarkan ID tertentu.

Implementasi ini memastikan bahwa admin memiliki kendali penuh untuk mengelola inventaris tiket langsung melalui antarmuka CMS yang telah terintegrasi dengan layanan Backend

c. Menambahkan untuk save file excel



```
1 ...
2 ...
3 ...
4 ...
5 ...
6 ...
7 ...
8 ...
9 ...
10 ...
11 ...
12 ...
13 ...
14 ...
15 ...
16 ...
17 ...
18 ...
19 ...
20 ...
21 ...
22 ...
23 ...
24 ...
25 ...
26 ...
27 ...
28 ...
29 ...
30 ...
31 ...
32 ...
33 ...
34 ...
35 ...
36 ...
37 ...
38 ...
39 ...
40 ...
41 ...
42 ...
43 ...
44 ...
45 ...
46 ...
47 ...
48 ...
49 ...
50 ...
51 ...
52 ...
53 ...
54 ...
55 ...
56 ...
57 ...
58 ...
59 ...
60 ...
61 ...
62 ...
63 ...
64 ...
65 ...
66 ...
67 ...
68 ...
69 ...
70 ...
71 ...
72 ...
73 ...
74 ...
75 ...
76 ...
77 ...
78 ...
79 ...
80 ...
81 ...
82 ...
83 ...
84 ...
85 ...
86 ...
87 ...
88 ...
89 ...
90 ...
91 ...
92 ...
93 ...
94 ...
95 ...
96 ...
97 ...
98 ...
99 ...
100 ...
```

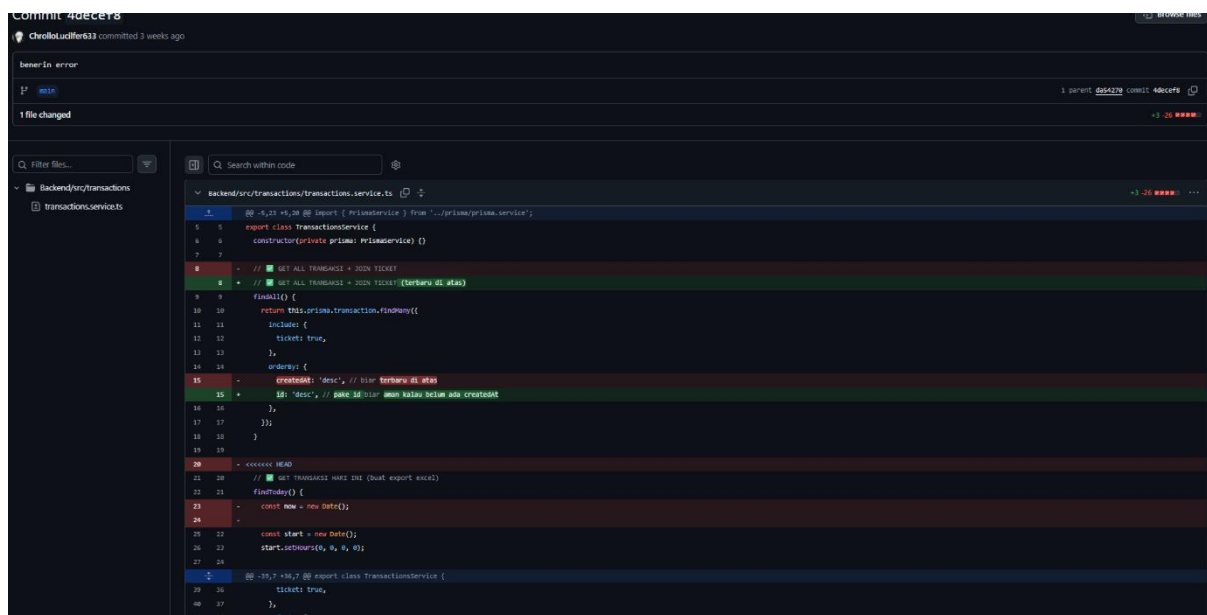
Pada tahap ini, saya menambahkan fitur untuk mengekspor data tiket ke dalam format file Excel pada sisi Frontend CMS. Untuk mendukung fitur ini, saya menginstal library exceljs dan file-saver melalui perintah npm install.

Saya mengimplementasikan logika ini di dalam tickets.service.ts dengan membuat fungsi saveToExcel. Fungsi ini bekerja dengan cara:

1. Membuat workbook dan worksheet baru menggunakan library ExcelJS.
2. Menyusun header kolom seperti 'ID', 'Name', dan 'Price'.
3. Memasukkan seluruh data tiket yang ada ke dalam baris-baris Excel.
4. Menghasilkan buffer data dan menyimpannya sebagai file bernama 'tickets.xlsx' menggunakan bantuan library FileSaver agar admin dapat mengunduh laporan data secara instan.

Penambahan fitur ini sangat krusial untuk memudahkan admin dalam mengelola data secara offline maupun untuk kebutuhan pelaporan eksternal

d. Benerin error



```
Commit 4dece78
Checkout: 4dece78 committed 3 weeks ago

benerin error

1 file changed

+3 -25 =====

Search within code

Backend/src/transactions
  transactions_services

1  @@ -4,22 +4,26 @@ import { PrismaService } from '../prisma/prisma.service';
2  export class TransactionService {
3  constructor(private prisma: PrismaService) {}
4
5  // GET ALL TRANSAKSI + JOIN TICKET
6  // GET ALL TRANSAKSI + JOIN TICKET (terbaru di atas)
7
8  findAll() {
9    return this.prisma.transaction.findMany({
10     include: {
11       ticket: true,
12     },
13     orderBy: {
14       createdAt: 'desc', // baru terbaru di atas
15     },
16     // id: 'desc', // baru id baru akan kalau belum ada createdAt
17   });
18 }
19
20 // ===== HEAD
21 // GET TRANSAKSI HARI INI (buat export excel)
22 findToday() {
23   const now = new Date();
24   const start = new Date();
25   start.setHours(0, 0, 0);
26   start.setMinutes(0, 0, 0);
27
28   // @@ -19,7 +16,7 @@ export class TransactionService {
29   //   ticket: true,
30   // },
31   // orderBy: {
```

Pada tahap ini, saya melakukan serangkaian perbaikan (bug fixing) untuk memastikan fitur-fitur pada CMS berjalan dengan stabil. Fokus utama perbaikan ini meliputi:

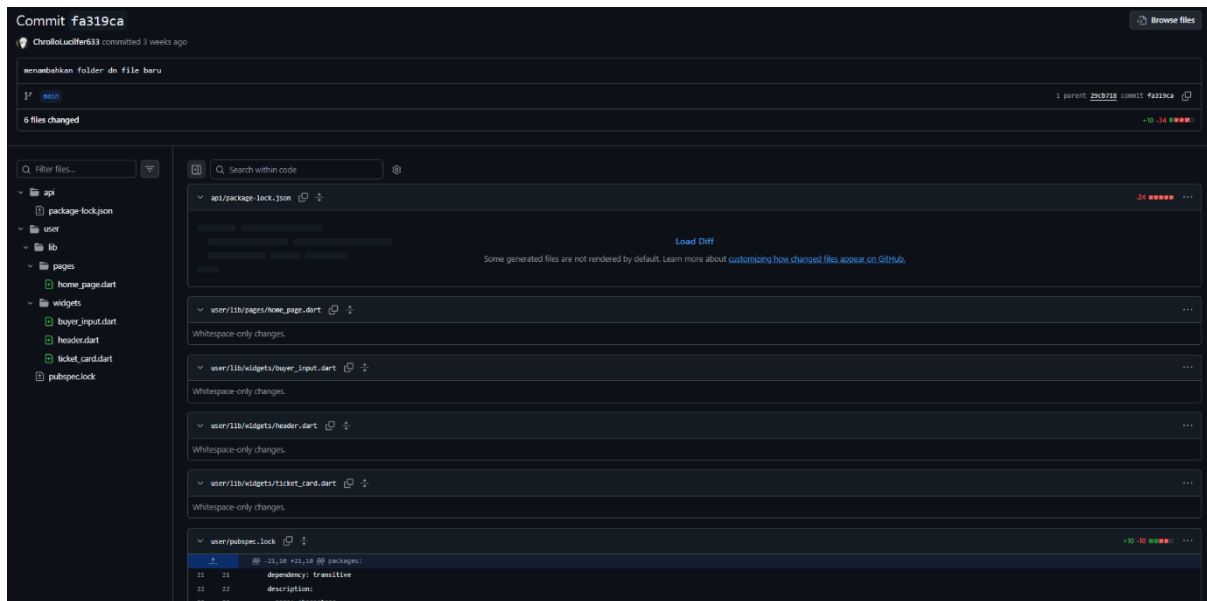
- Penanganan Error pada Export Excel: Saya memperbaiki logika pada fungsi saveToExcel di tickets.service.ts. Perbaikan dilakukan dengan menambahkan blok try-catch untuk menangkap potensi error saat proses generate file, serta memastikan data yang di-export sudah dalam format yang benar agar tidak menyebabkan aplikasi crash.
- Validasi Data Kosong: Saya menambahkan pengecekan kondisi untuk memastikan fungsi export hanya berjalan jika data tiket tersedia. Jika data kosong, sistem akan memberikan peringatan atau mencegah proses download.

- Optimasi Library: Saya memastikan pemanggilan library exceljs dan file-saver dilakukan secara benar sesuai dengan standar modul TypeScript di NestJS untuk menghindari error 'undefined' saat aplikasi dijalankan di lingkungan produksi.

Langkah perbaikan ini dilakukan untuk meningkatkan pengalaman pengguna (UX) dan menjamin keandalan sistem dalam mengolah laporan data

8. Commits on April 18, 2026

a. Menambahkan folder dan file baru



Pada tahap pengembangan ini, saya memperluas fungsionalitas Dashboard Admin dengan menambahkan folder dan file baru untuk modul Transactions. Langkah ini meliputi pembuatan struktur folder transactions di dalam frontend-cms/src, yang terdiri dari:

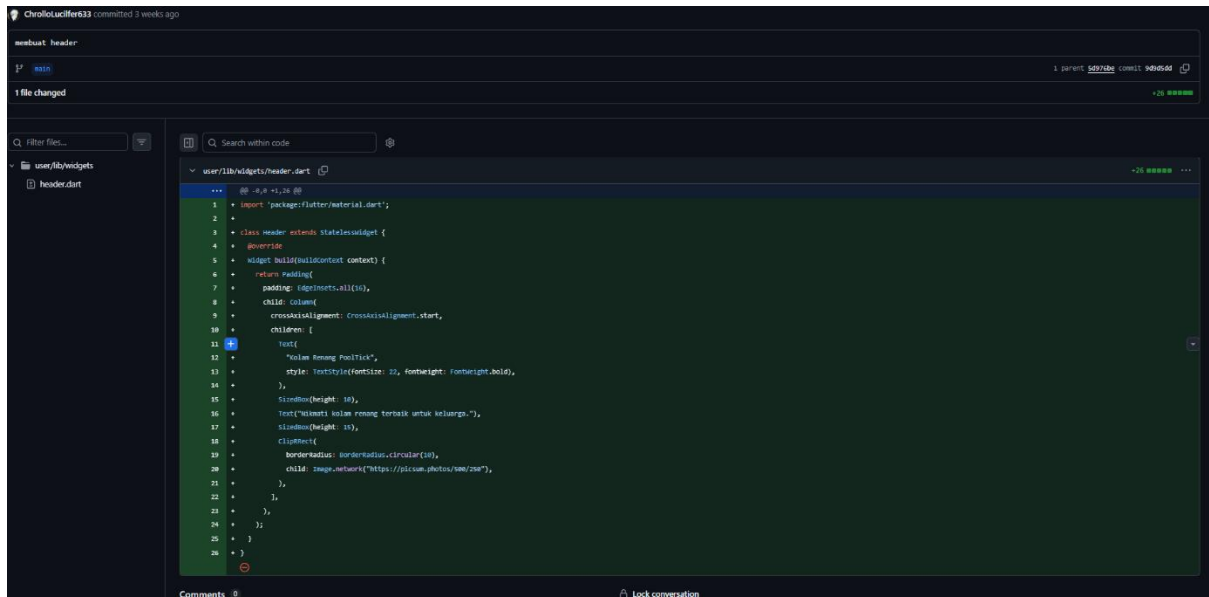
- `transactions.module.ts`: Untuk mendaftarkan dependensi dan penyedia layanan transaksi.
- `transactions.service.ts`: Untuk menangani komunikasi data dengan API Backend, termasuk fungsi untuk mengambil riwayat transaksi.
- Integrasi Modul: Saya juga mendaftarkan `TransactionsModule` ke dalam file utama `cms.module.ts` agar fitur ini dapat diakses secara global di dalam aplikasi CMS.

Penambahan file-file baru ini bertujuan agar admin dapat memantau seluruh aktivitas transaksi secara *real-time*, melengkapi fitur manajemen tiket yang sudah dibuat sebelumnya

[illegible]

The screenshot shows a commit diff interface. At the top, the commit hash '5d976be' and author 'ChroloLucifers33' are displayed. Below this, a file tree on the left shows the project structure with 'userlib/services' selected. The main area shows a diff for the file 'api.dart'. The diff highlights changes to the 'getTickets' method, including the addition of a 'Future<List<dynamic>>' return type, a 'final response' variable, a 'uri.parse' call for the tickets URL, and a 'throw exception' statement for a 'sagal load ticket' error. The diff also shows the addition of a 'class ApiService' and a 'getTickets()' method that returns a decoded JSON response. The bottom of the diff shows the 'beginTransaction' method with headers for 'Content-Type: application/json'.

d. Membuat header

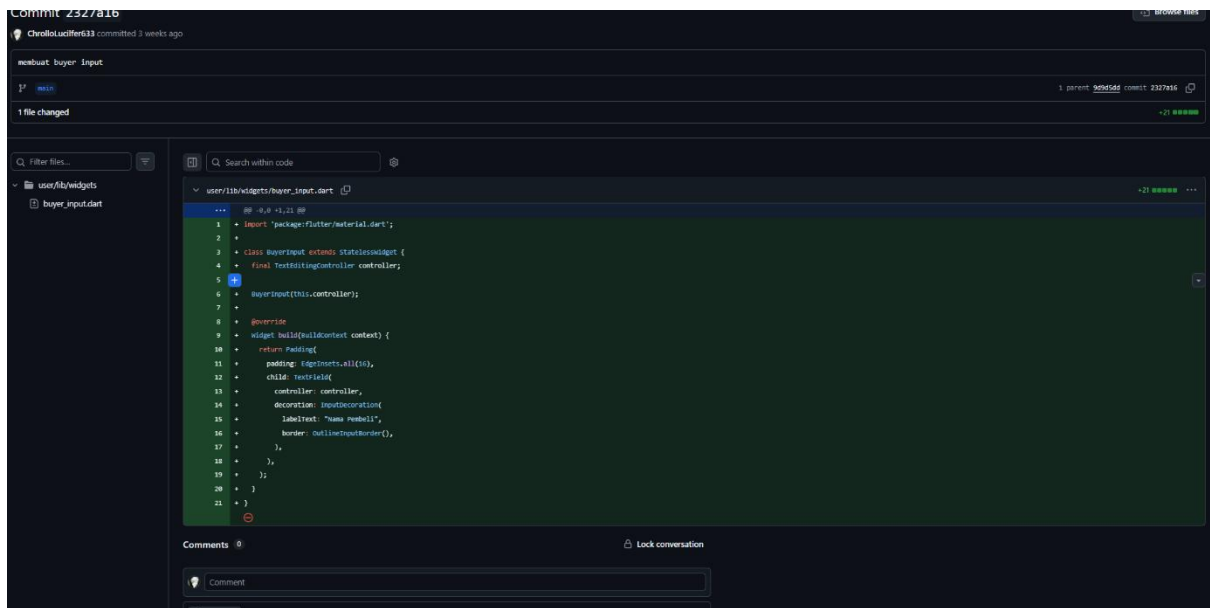


Pada tahap ini, saya fokus pada pengembangan antarmuka (UI) dengan membuat komponen Header pada Frontend CMS. Implementasi ini dilakukan pada file index.html dan file layout terkait dengan beberapa poin perubahan sebagai berikut:

- **Penyusunan Navigasi:** Saya menambahkan elemen navigasi menggunakan Bootstrap untuk menciptakan header yang responsif. Header ini mencakup identitas aplikasi (Brand) dan menu navigasi utama untuk berpindah antar halaman (Tiket dan Transaksi).
- **Integrasi Ikon:** Saya memanfaatkan library Bootstrap Icons untuk menambahkan elemen visual pada header agar antarmuka lebih intuitif bagi pengguna (admin).
- **Struktur Container:** Saya merapikan struktur pembungkus (*container*) agar konten utama di bawah header memiliki margin yang konsisten dan terlihat rapi di berbagai ukuran layar.

Langkah ini memastikan bahwa CMS tidak hanya fungsional di sisi data, tetapi juga memiliki navigasi yang jelas dan user-friendly bagi pengelola system

e. Membuat buyer input

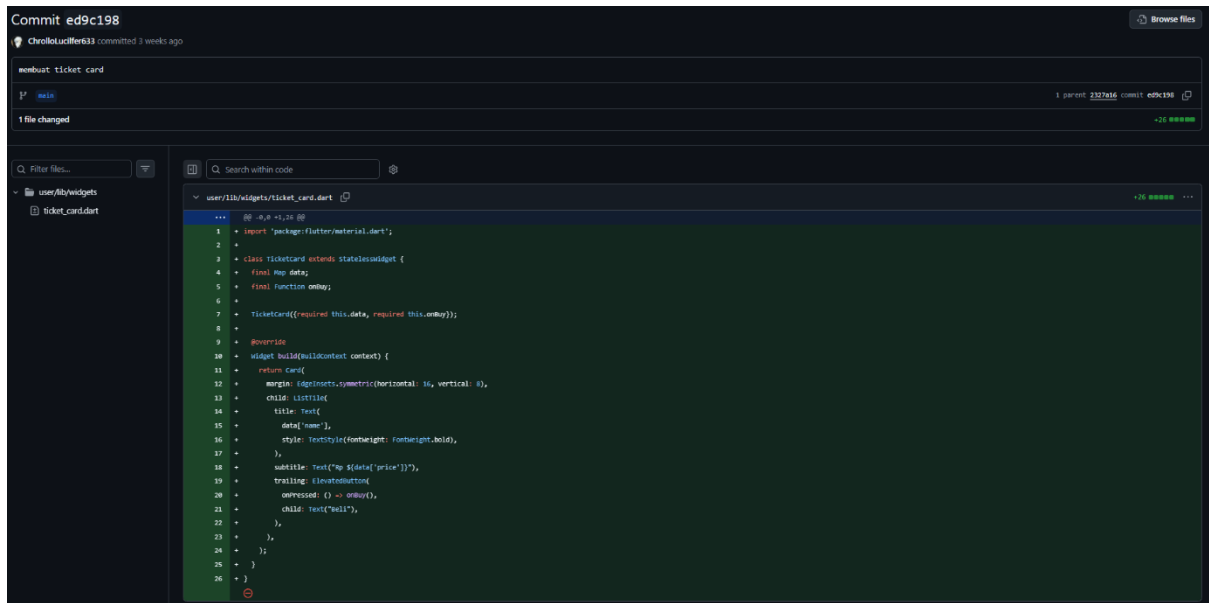


Pada tahap ini, saya memperbaiki modul transaksi dengan menambahkan fitur Buyer Input. Perubahan ini bertujuan agar sistem tidak hanya mencatat tiket yang terjual, tetapi juga identitas pembelinya. Langkah-langkah yang saya lakukan meliputi:

- Pembaruan Skema Data: Menambahkan field buyer (nama pembeli) pada logika pembuatan transaksi di sisi Frontend CMS dan Backend
- Integrasi Form: Memperbarui antarmuka input pada CMS agar admin dapat memasukkan nama pembeli saat mencatat transaksi baru secara manual
- Sinkronisasi Service: Menyesuaikan fungsi create pada `transactions.service.ts` agar dapat menerima dan menyimpan data buyer ke dalam database bersamaan dengan `ticketId` dan data lainnya

Dengan adanya fitur ini, laporan transaksi menjadi lebih lengkap dan akuntabel karena setiap penjualan tiket kini memiliki data pemilik atau pembeli yang jelas

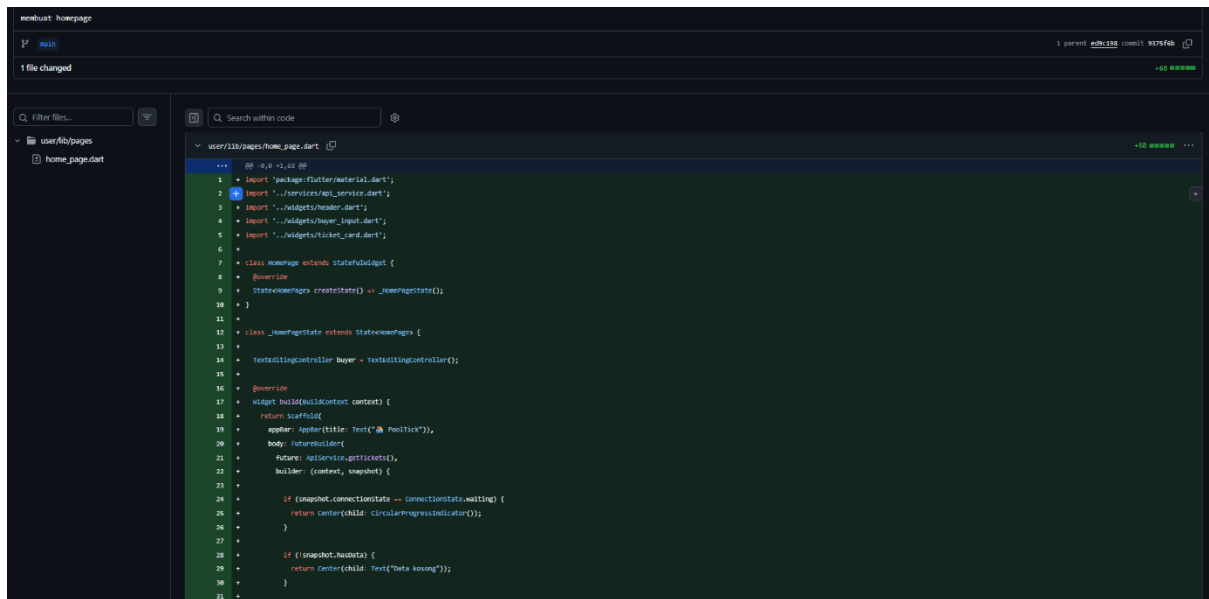
f. Membuat ticket card



Pada tahap ini, saya menyempurnakan sisi antarmuka untuk menampilkan daftar transaksi yang telah berhasil dicatat. Fokus utama dalam perubahan ini adalah:

- Integrasi Data Pembeli: Memastikan kolom 'Buyer' atau nama pembeli muncul dengan benar pada tabel daftar transaksi di dashboard admin.
- Refactoring Template: Saya memperbarui file index.html dan logika rendering pada CMS agar dapat memetakan (*mapping*) data transaksi dari API secara dinamis ke dalam tabel.
- Verifikasi Data Relasional: Saya memastikan bahwa selain nama pembeli, informasi terkait (seperti nama tiket dan harga) yang ditarik melalui relasi database juga tampil secara akurat di sisi Frontend.

g. Membuat homepage

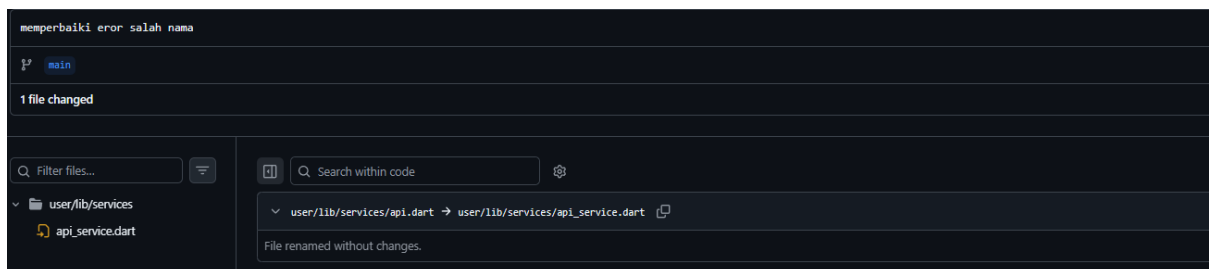


Pada tahap ini, saya mengimplementasikan halaman utama (Homepage) untuk Dashboard Admin pada Frontend CMS. Fokus pengerjaan pada tahap ini meliputi:

- Pembuatan Struktur Homepage: Saya menyusun file index.html sebagai halaman landas (*landing page*) bagi admin setelah melakukan login. Halaman ini berfungsi sebagai pusat navigasi untuk mengarahkan admin ke manajemen tiket maupun riwayat transaksi.
- Integrasi Komponen Header: Saya menyempurnakan komponen header agar tampil secara konsisten di halaman utama. Header ini dilengkapi dengan link navigasi yang memungkinkan admin berpindah antar modul dengan cepat tanpa perlu memuat ulang seluruh halaman secara manual.
- Layouting Responsif: Menggunakan grid sistem dari Bootstrap, saya memastikan tata letak homepage tetap rapi dan informatif saat diakses melalui berbagai perangkat dengan ukuran layar yang berbeda.

Tahap ini menandai selesainya kerangka dasar antarmuka pengguna pada sisi CMS, di mana seluruh fitur utama kini telah terhubung dalam satu navigasi yang terpadu

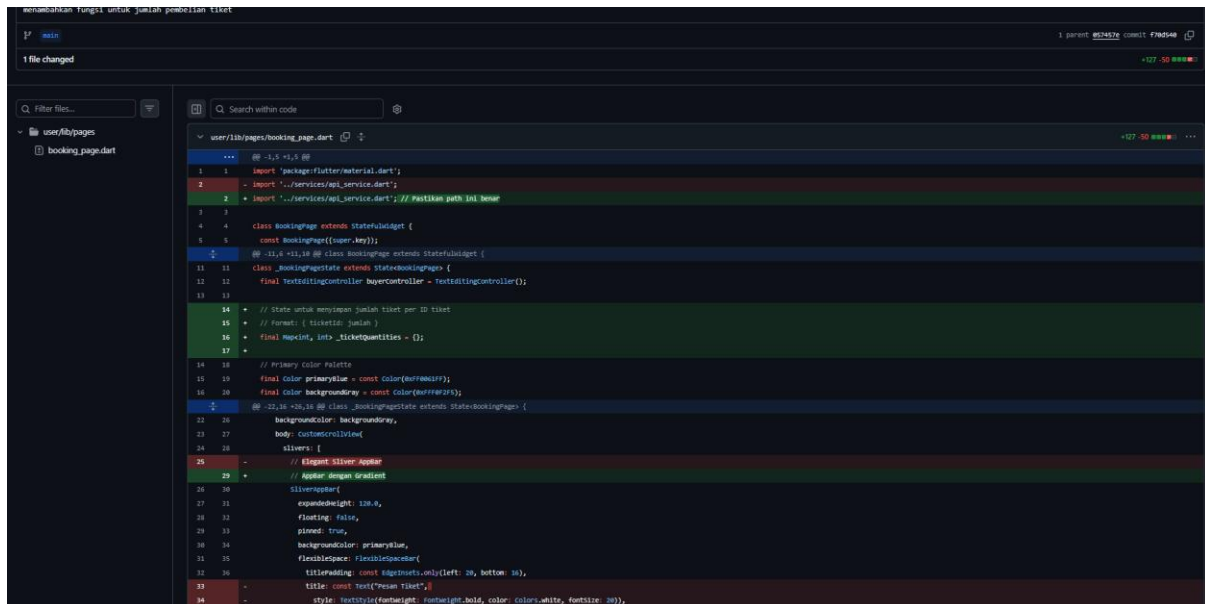
h. Memperbaiki error salah nama



Di sini saya melakukan perbaikan nama

9. Commits on April 20, 2026

a. Menambahkan fungsi untuk jumlah pembeli tiket



```
menambahkan fungsi untuk jumlah pembelian tiket
1 parent: 604872 commit: 776554
1 file changed

user/lib/pages/booking_page.dart
1 1 import 'package:flutter/material.dart';
2 2 import '../services/api_service.dart';
3 3 import '../services/api_service.dart'; // Pastikan path ini benar
4 4
5 5 class BookingPage extends StatefulWidget {
6 6   const BookingPage({super.key});
7 7
8 8   @override
9 9   State<BookingPage> createState() {
10 10    return _BookingPageState();
11 11   }
12 12
13 13   class _BookingPageState extends State<BookingPage> {
14 14     final TextEditingController _controller = TextEditingController();
15 15
16 16     // State untuk menyimpan jumlah tiket per ID tiket
17 17     // Format: { tiketId: jumlah }
18 18     final Map<int, int> _ticketQuantities = {};
19 19
20 20     // Primary Color Palette
21 21     final Color primaryBlue = Color(0xFF0070C0);
22 22     final Color backgroundColor = Color(0xFFFFF0F5);
23 23
24 24     @override
25 25     Widget build(BuildContext context) {
26 26       return Scaffold(
27 27         backgroundColor: backgroundColor,
28 28         appBar: AppBar(
29 29           title: Text('Pemesanan Tiket'),
30 30           backgroundColor: primaryBlue,
31 31           elevation: 0,
32 32         ),
33 33         body: ListView(
34 34           children: [
35 35             // Header
36 36             Container(
37 37               padding: 10,
38 38               child: Text(
39 39                 'Pemesanan Tiket',
40 40                 style: TextStyle(
41 41                   color: primaryBlue,
42 42                   fontWeight: FontWeight.bold,
43 43                   fontSize: 18,
44 44                 ),
45 45             ),
46 46             // List of tickets
47 47             ListView.builder(
48 48               itemCount: 5,
49 49               itemBuilder: (context, index) {
50 50                 // Mock data for tickets
51 51                 final ticketId = index + 1;
52 52                 final price = 100000;
53 53                 final quantity = 1;
54 54                 // Add to state
55 55                 _ticketQuantities[ticketId] = quantity;
56 56                 // Build ticket item
57 57                 return Card(
58 58                   margin: 10,
59 59                   child: ListTile(
60 60                     title: 'Tiket $ticketId',
61 61                     subtitle: 'Harga: Rp. $price',
62 62                     trailing: Text('Jumlah: $quantity'),
63 63                   ),
64 64                 );
65 65               },
66 66             ),
67 67           ],
68 68         ),
69 69       );
70 70     }
71 71   }
72 72 }
73 73
```

Pada tahap pengembangan ini, saya melakukan pembaruan signifikan pada modul transaksi dengan menambahkan fungsi Kuantitas Pembelian (Quantity). Fitur ini memungkinkan pengguna atau admin untuk menginput jumlah tiket yang dibeli dalam satu kali transaksi. Perubahan teknis yang saya lakukan meliputi:

- **Pembaruan Logika Backend:** Pada `transactions.service.ts`, saya memodifikasi fungsi `create` agar dapat menerima input `quantity`. Sistem kini akan melakukan validasi stok berdasarkan jumlah yang diminta (`quantity`), bukan lagi hanya satu per satu.
- **Kalkulasi Otomatis:** Saya menambahkan logika untuk menghitung total harga secara otomatis dengan mengalikan harga satuan tiket dengan kuantitas yang diinput (`price * quantity`).
- **Sinkronisasi Stok:** Logika pengurangan stok tiket di database juga diperbarui agar berkurang sesuai dengan jumlah kuantitas yang dibeli dalam transaksi tersebut.
- **Integrasi UI CMS:** Pada sisi Frontend CMS, saya menambahkan kolom input 'Quantity' pada form transaksi dan memperbarui tabel daftar transaksi agar menampilkan kolom jumlah tiket serta total harga yang harus dibayar.

Dengan penambahan fitur ini, sistem menjadi lebih fleksibel dan mampu menangani skenario pembelian tiket dalam jumlah banyak secara akurat dan efisien.

```

1 // ignore_for_file: unused_import, unused_element
2
3 import 'package:flutter/material.dart';
4
5 class BookingPage extends StatefulWidget {
6   const BookingPage({Key? key}) : super(key: key);
7
8   @override
9   _BookingPageState createState() => _BookingPageState();
10 }
11
12 class _BookingPageState extends State<BookingPage> {
13   final TextEditingController _buyerController = TextEditingController();
14   final Map<int, int> _cart = {};
15   final List<Ticket> _tickets = [];
16
17   // State untuk menyimpan jumlah tiket per 10 tiket
18   // Format: { ticketId: jumlah }
19   final Map<int, int> _ticketQuantities = {};
20
21   // Primary color palette
22   // Palette warna modern
23   final Color primaryViolet = const Color(0xFF91587F);
24   final Color background = const Color(0xFFFFF5F5);
25   final Color darkTeal = const Color(0xFF00796B);
26   final Color accentBlue = const Color(0xFF00A08A);
27   final Color bgPaper = const Color(0xFFFFF5F5);
28   final Color surfaceWhite = Colors.white;
29   final Color textDark = const Color(0xFF1E282C);
30
31   // Method to calculate total price
32   int _calculateTotal() {
33     int total = 0;
34     _cart.forEach((id, qty) {
35       var ticket = _tickets.firstWhere((t) => t.id == id, orElse: () => null);
36       if (ticket != null) total += (ticket['price'] as int) * qty;
37     });
38     return total;
39   }
40
41   @override
42   Widget build(BuildContext context) {
43     int totalBuyer = _calculateTotal();
44
45     return Scaffold(
46       appBar: AppBar(
47         title: Text('Booking Page'),
48       ),
49       body: Padding(
50         padding: const EdgeInsets.all(16.0),
51         child: Column(
52           children: [
53             Text('Buyer Name:'),
54             TextField(
55               controller: _buyerController,
56             ),
57             SizedBox(height: 10),
58             Text('Available Tickets:'),
59             ListView.builder(
60               itemCount: _tickets.length,
61               itemBuilder: (context, index) {
62                 final ticket = _tickets[index];
63                 return Card(
64                   margin: 10,
65                   child: ListTile(
66                     title: Text('Ticket ID: ${ticket.id}'),
67                     subtitle: Text('Price: ${ticket.price}'),
68                     trailing: Text('Quantity: ${_ticketQuantities[ticket.id] ?? 0}'),
69                   ),
70                 );
71               },
72             ),
73             Text('Total Price: $totalBuyer'),
74           ],
75         ),
76       ),
77     );
78   }
79 }

```

- **Pembaruan Entitas:** Saya menambahkan kolom baru yaitu quantity ke dalam entitas Transaction dengan tipe data number. Hal ini memastikan bahwa jumlah tiket yang dibeli tersimpan secara permanen di dalam database PostgreSQL.
- **Optimalisasi Penyimpanan:** Saya memperbarui fungsi create pada service agar field quantity yang dikirim dari frontend dipetakan dengan benar ke dalam database.
- **Integritas Data:** Dengan pendaftaran kolom quantity di level entitas, sistem kini dapat melakukan audit data yang lebih akurat, di mana setiap record transaksi mencatat detail jumlah tiket tanpa mengandalkan kalkulasi sementara di sisi client.

10. Commits on 23 Mei, 2026

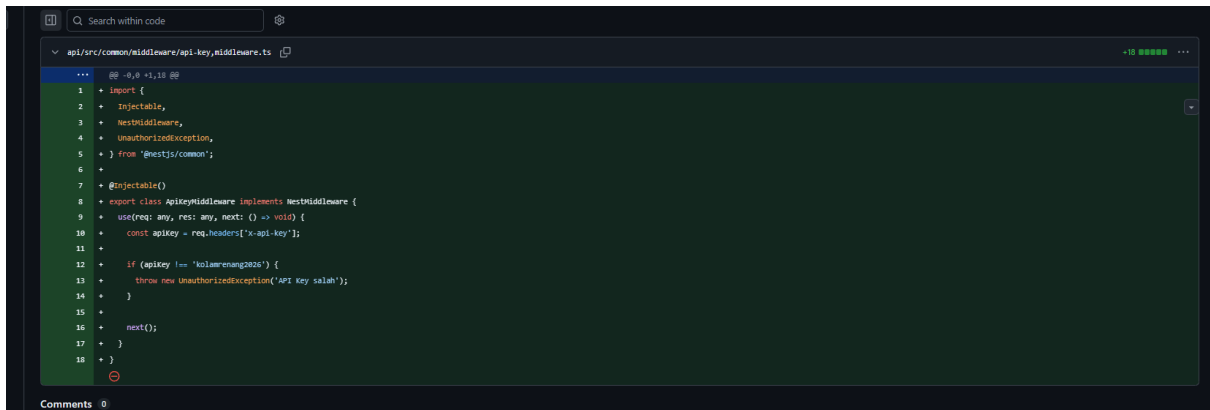
The screenshot shows the Visual Studio Code interface with the 'PORTS' tab selected. The 'PORTS' tab is highlighted with a blue circle containing the number '1'. Below the tab bar, there is a table with the following columns: 'Port', 'Forwarded Address', 'Running Process', 'Visibility', and 'Origin'. A single port is listed: '52338', with a forwarded address of 'https://h4fmj7zt-52338.as...' and visibility set to 'Public'. An 'Add Port' button is located below the table.

Port	Forwarded Address	Running Process	Visibility	Origin
52338	https://h4fmj7zt-52338.as...		Public	User Forwarded

Disini mengubah visibility port 52338 menjadi public supaya dapat dibuka di handphone

11. Commits on 10 juni, 2026

a. Feat : add api key middleware



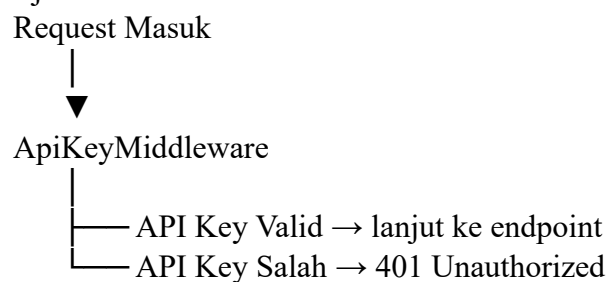
```
1  + import {  
2  +   Injectable,  
3  +   NestMiddleware,  
4  +   UnauthorizedException,  
5  + } from '@nestjs/common';  
6  +  
7  + @Injectable()  
8  + export class ApiKeyMiddleware implements NestMiddleware {  
9  +   use(req: any, res: any, next: () => void) {  
10 +     const apiKey = req.headers['x-api-key'];  
11 +  
12 +     if (apiKey !== 'kolawenang2026') {  
13 +       throw new UnauthorizedException('API Key salah');  
14 +     }  
15 +  
16 +     next();  
17 +   }  
18 + }
```

Pada tahap ini ditambahkan middleware baru Bernama ApiKeyMiddleware. Middleware digunakan untuk memeriksa apakah request yang masuk memiliki API Key yang valid atau tidak

Fungsi :

- Menambahkan system keamanan pada backend.
- Mencegah akses API tanpa izin

Cara Kerja :



b. Feat : register api key middleware

```
api/src/app.module.ts
1 - import { Module } from '@nestjs/common';
2 + import { MiddlewareConsumer, Module, NestModule } from '@nestjs/common';
3 + import { ApikeyMiddleware } from '../common/middleware/api-key.middleware';
4
5 import { AppController } from '../app.controller';
6 import { AppService } from '../app.service';
7
8 @Module({
9   imports: [
10     - PrismaModule,
11     + PrismaModule,
12     TicketsModule,
13     TransactionsModule,
14   ],
15   controllers: [AppController],
16   providers: [AppService],
17 })
18 - export class AppModule {}
19
20 + export class AppModule implements NestModule {
21 +   configure(consumer: MiddlewareConsumer) {
22 +     consumer.apply(ApikeyMiddleware).forRoutes(
23 +       'tickets',
24 +       'transactions',
25 +     );
26 +   }
27 + }
28 + }
```

Middleware yang sebelumnya dibuat kemudia didaftarkan ke APPModule.

Fungsi :

- Menghubungkan middleware ke endpoint backend.
- Membuat middleware aktif Ketika endpoint dipanggil.

Endpoint yang Dilindungi

/tickets

/transactions

c. Feat : protect admin routes with api key

```
api/src/app.module.ts
1 - import { MiddlewareConsumer, Module, NestModule } from '@nestjs/common';
2 + import {
3 +   MiddlewareConsumer,
4 +   Module,
5 +   NestModule,
6 +   RequestMethod,
7 + } from '@nestjs/common';
8
9 import { ApikeyMiddleware } from '../common/middleware/api-key.middleware';
10
11 import { AppController } from '../app.controller';
12
13 @Module({
14   imports: [PrismaModule],
15 })
16 export class AppModule implements NestModule {
17   configure(consumer: MiddlewareConsumer) {
18     consumer.apply(ApikeyMiddleware).forRoutes(
19       - 'tickets',
20       - 'transactions',
21
22       // ADMIN TICKETS
23       {
24         path: 'tickets',
25         method: RequestMethod.POST,
26       },
27       {
28         path: 'tickets/:id',
29         method: RequestMethod.PATCH,
30       },
31       {
32         path: 'tickets/:id',
33         method: RequestMethod.DELETE,
34       },
35
36       // ADMIN TRANSACTIONS
37       {
38         path: 'transactions',
39         method: RequestMethod.GET,
40       },
41     );
42   }
43 }
```

Backend diperbarui agar hanya route admin tertentu yang memerlukan API key.

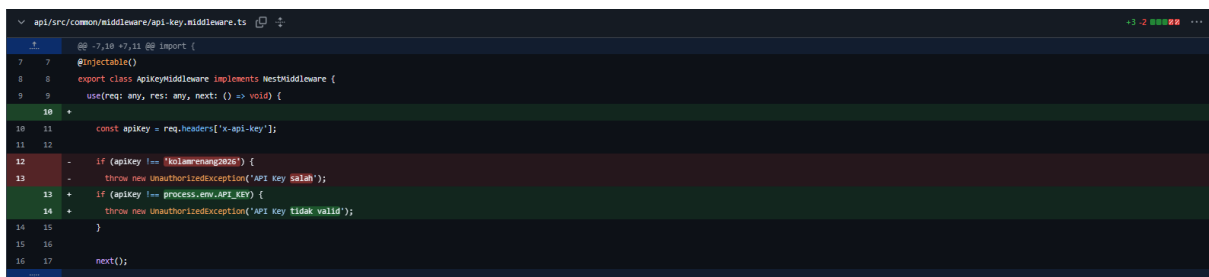
Route yang diamankan :

POST /tickets
PATCH /tickets/:id
DELETE /tickets/:id
GET /transactions
POST /transactions
DELETE /transactions/:id

Fungsi :

- Endpoint admin tidak bisa diakses sembarangan
- CMS tetap dapat digunakan dengan menambahkan header API Key.

d. Feat : move api key to environment variables



```
api/src/common/middleware/api-key.middleware.ts
import {
  Injectable
} from '@nestjs/common';
import {
  NextFunction,
  Request,
  Response
} from 'express';
import {
  UnauthorizedException
} from '@nestjs/common';

@Injectable()
export class ApiKeyMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: NextFunction): void {
    const apiKey = req.headers['x-api-key'];

    if (apiKey !== 'kolamrenang2026') {
      throw new UnauthorizedException('API key salah');
    }
    if (apiKey !== process.env.API_KEY) {
      throw new UnauthorizedException('API key tidak valid');
    }

    next();
  }
}
```

API Key dipindahkan ke file .env

Sebelum : `const API_KEY = "kolamrenang2026"`

Sesudah : `API_KEY=*****`

Fungsi :

- Membuat API Key lebih aman
- Mempermudah penggantian API Key tanpa mengubah source code

e. Update env



```
api/.env copy
DATABASE_URL="postgres://postgres:postgres@localhost:5432/pooltick"
API_KEY=kolamrenang2026
```

Melakukan pembaruan file .env untuk menyesuaikan konfigurasi backend.

Isi konfigurasi

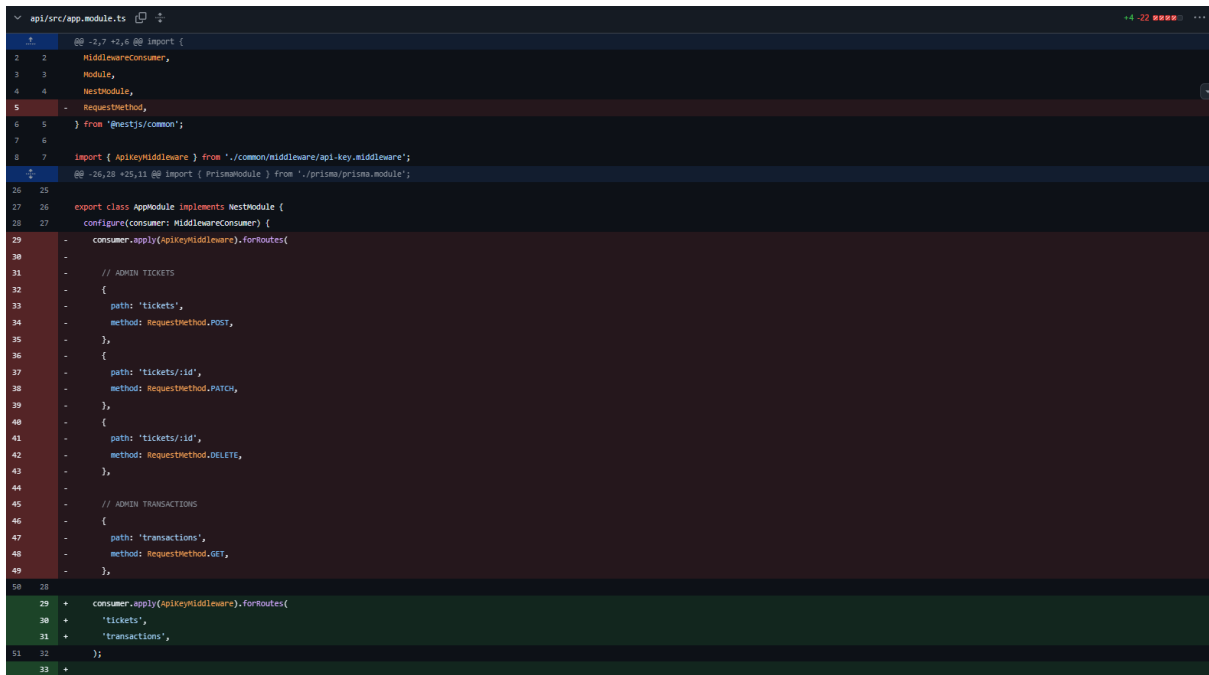
`DATABASE_URL=...`

`API_KEY=...`

Fungsi :

- Menghubungkan backend ke database PostgreSQL
- Menyimpan konfigurasi penting backend.

f. Kunci semua endpoint



```
1  // api/src/app.module.ts
2  @ -2,7 +2,6 @@ import {
3    MiddlewareConsumer,
4    Module,
5    NestModule,
6    RequestMethod,
7  } from '@nestjs/common';
8
9  import { ApiKeyMiddleware } from '../common/middleware/api-key.middleware';
10
11  @ -26,28 +25,11 @@ import { PrismaModule } from '../prisma/prisma.module';
12
13  export class AppModule implements NestModule {
14    configure(consumer: MiddlewareConsumer) {
15      consumer.apply(ApiKeyMiddleware).forRoutes(
16        // ADMIN TICKETS
17        [
18          {
19            path: 'tickets',
20            method: RequestMethod.POST,
21          },
22          {
23            path: 'tickets/:id',
24            method: RequestMethod.PATCH,
25          },
26          {
27            path: 'tickets/:id',
28            method: RequestMethod.DELETE,
29          },
30          // ADMIN TRANSACTIONS
31          [
32            {
33              path: 'transactions',
34              method: RequestMethod.GET,
35            },
36          ],
37        ],
38      );
39    }
40  }
```

Seluruh endpoint backend diperbarui agar wajib menggunakan API Key.

Fungsi :

- Semua request ke backend menjadi lebih aman.
- Request tanpa API Key akan otomatis ditolak.

Response error :

```
{
  "statusCode": 401,
  "message": "API Key tidak valid"
}
```